

MAGISTRAL

02 Definiciones dirigidas por la sintaxis

M.Sc. Luis Fernando Espino Barrios
2026

Una definición dirigida por la sintaxis es una gramática libre de contexto junto con atributos y reglas.

Además, especifica los valores de los atributos mediante la asociación de las reglas semánticas con las producciones gramaticales.

Regla semántica

PRODUCTION

$E \rightarrow E_1 + T$

SEMANTIC RULE

$E.code = E_1.code \parallel T.code \parallel '+'$

(5.1)

$E \rightarrow E_1 + T \{ \text{print '+'} \}$

(5.2)

Acción semántica

Más legible y más útil
para las
especificaciones

PRODUCTION

$E \rightarrow E_1 + T$

SEMANTIC RULE

$E.code = E_1.code \parallel T.code \parallel '+'$

(5.1)

$E \rightarrow E_1 + T \{ \text{print '+'} \}$

(5.2)

Más eficientes para las
implementaciones

Los atributos se asocian con símbolos gramaticales; y las reglas se asocian con producciones.

Un atributo sintetizado para un no terminal A en un nodo N de un árbol sintáctico se define mediante una regla semántica asociada con la producción de N.

Un atributo heredado para un no terminal B en el nodo N de un árbol de análisis sintáctico se define mediante una regla semántica asociada con la producción en el padre de N.

Evaluación de una definición

Es la manera de evaluar los atributos en todos los hijos de un nodo, mediante el árbol sintáctico.

Un árbol sintáctico que muestra los valores de sus atributos se conoce como anotado.

Ejemplo 1

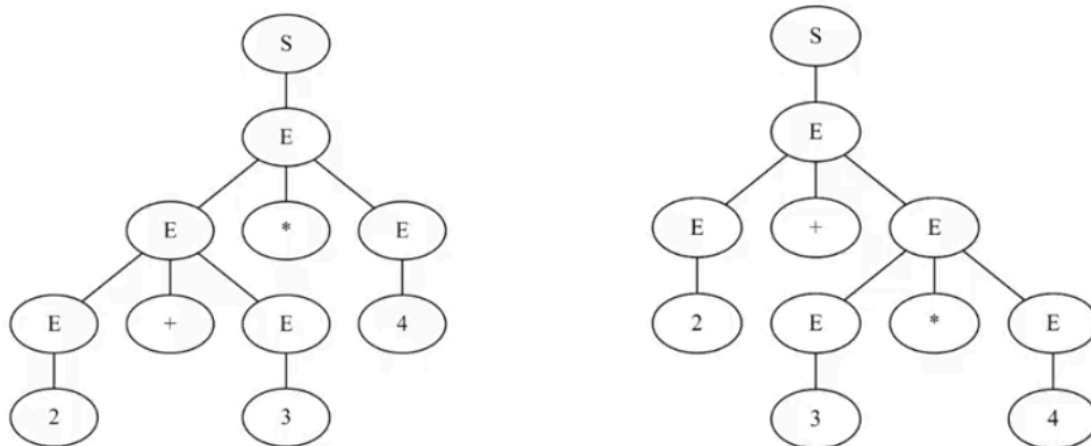
Construir el árbol de análisis sintáctico anotado, basado en la gramática antes vista, evaluando la entrada: $3*5+4n$

Ejemplo 1

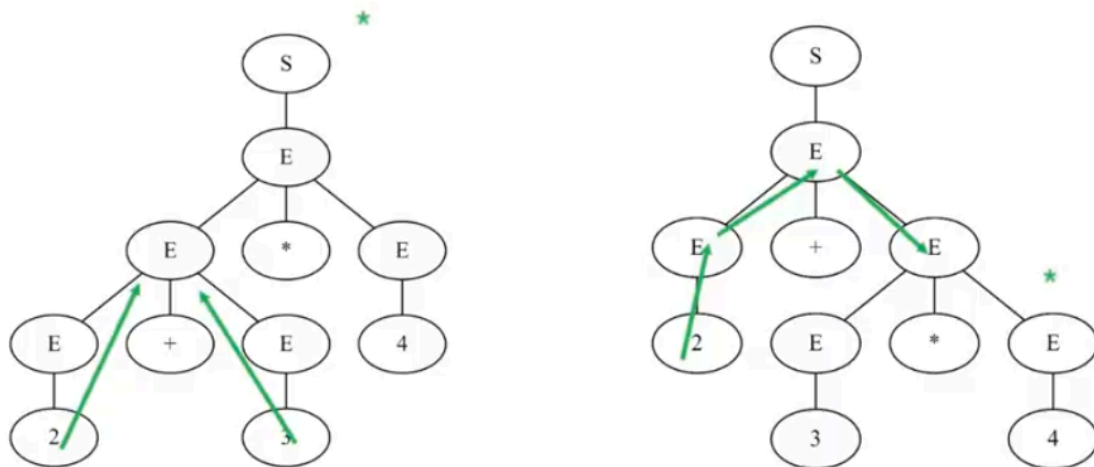
PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

Figure 5.1: Syntax-directed definition of a simple desk calculator

Dos árboles de derivación distintos



Dos árboles de derivación distintos



Ejemplo 1

PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

Figure 5.1: Syntax-directed definition of a simple desk calculator

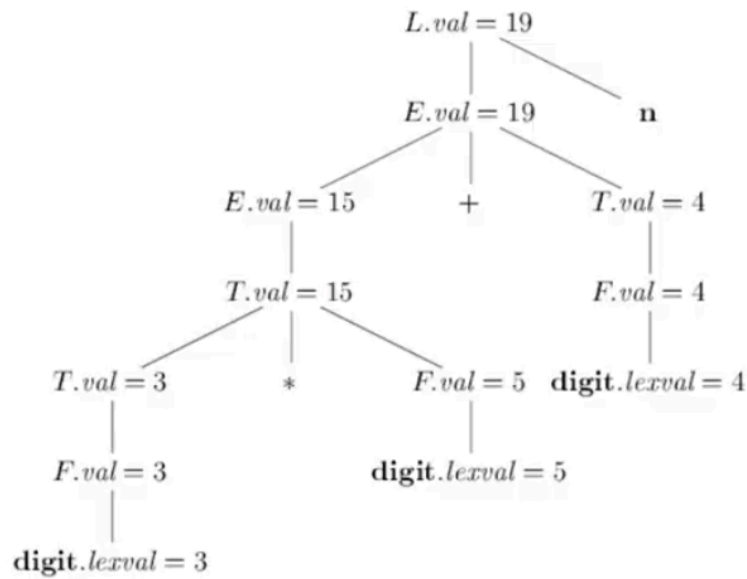


Figure 5.3: Annotated parse tree for $3 * 5 + 4 \mathbf{n}$

PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

Resumen - Definiciones Dirigidas por la Sintaxis

1 ¿Qué es una Definición Dirigida por la Sintaxis?

Una **Definición Dirigida por la Sintaxis (DDS o SDD)** es:

Una **gramática libre de contexto (GLC)** +
atributos +
reglas semánticas

En palabras simples:

- No solo describes cómo se forma una expresión (sintaxis),
- sino también **qué significa** y **cómo se calcula su valor** (semántica).

Ejemplo:

No solo parsear $3 * 5 + 4$, sino también **calcular que vale 19**.

2 Componentes de una SDD

a) Gramática Libre de Contexto (GLC)

Es lo típico de compiladores:

$E \rightarrow E + T$
 $T \rightarrow T * F$
 $F \rightarrow \text{digit}$

Describe la **estructura** de las expresiones.

b) Atributos

Los atributos son **valores asociados a los símbolos gramaticales**.

Ejemplos:

- $E.val$
- $T.val$

- `F.val`
- `digit.lexval`

Sirven para:

- Calcular resultados
- Guardar tipos
- Guardar direcciones
- Construir árboles
- Generar código

c) Reglas Semánticas

Son fórmulas que dicen **cómo calcular los atributos**.

Ejemplo clásico:

Producción:

$E \rightarrow E1 + T$

Regla semántica:

$E.val = E1.val + T.val$

O sea:

👉 El valor de E es la suma de los valores de E1 y T.

3 Regla Semántica vs Acción Semántica

Esto también lo viste:

- ♦ **Regla semántica**

- Más clara
- Más declarativa
- Mejor para especificar

Ejemplo:

```
E.val = E1.val + T.val
```

♦ Acción semántica

- Código embebido
- Más eficiente
- Usada en implementación real

Ejemplo (en parser):

```
{ E.val = E1.val + T.val; }
```

👉 En clase y teoría se usan **reglas semánticas**.

👉 En implementaciones reales, **acciones semánticas**.

4 Tipos de Atributos

● Atributos Sintetizados (Synthesized)

Se calculan:

⬆ De los hijos hacia el padre

Ejemplo:

```
F → digit
F.val = digit.lexval
```

El `digit` le pasa su valor a `F`.

Son los MÁS comunes.

Atributos Heredados (Inherited)

Se pasan:

 Del padre o hermanos hacia un nodo

Sirven para:

- Pasar contexto
- Acumular valores
- Manejar listas
- Manejar expresiones sin recursividad izquierda

Ejemplo conceptual:

$T'.inh = T.val$

Evaluación de una SDD

Cuando tienes el árbol sintáctico:

Árbol de análisis sintáctico

+

Atributos evaluados

 Árbol sintáctico anotado

Árbol sintáctico anotado

Es un árbol donde cada nodo tiene cosas como:

$E.val = 19$

$T.val = 15$

$F.val = 3$

Eso es justo lo que aparece en tu imagen:

Figure 5.3: Annotated parse tree for $3 * 5 + 4 n$

6 Ejemplo: $3 * 5 + 4 n$

Producciones típicas (como en tus capturas):

$L \rightarrow E n$

$L.val = E.val$

1.

$E \rightarrow E1 + T$

$E.val = E1.val + T.val$

2.

$E \rightarrow T$

$E.val = T.val$

3.

$T \rightarrow T1 * F$

$T.val = T1.val * F.val$

4.

$T \rightarrow F$

$T.val = F.val$

5.

$F \rightarrow (E)$

$F.val = E.val$

6.

$F \rightarrow \text{digit}$

$F.\text{val} = \text{digit}.\text{lexval}$

7.

Evaluación paso a paso

Para:

$3 * 5 + 4$

- $3 * 5 = 15$
- $15 + 4 = 19$

Entonces en el árbol:

- $F.\text{val} = 3$
- $F.\text{val} = 5$
- $T.\text{val} = 15$
- $F.\text{val} = 4$
- $E.\text{val} = 19$
- $L.\text{val} = 19$

Eso explica el árbol donde ves:

$L.\text{val} = 19$
 $E.\text{val} = 19$
 $T.\text{val} = 15$
 $F.\text{val} = 3$
 $\text{digit}.\text{lexval} = 3$

7 Dos árboles de derivación distintos

Esto muestra **ambigüedad**:

👉 La misma expresión puede tener
🌳 dos árboles de derivación diferentes

Pero:

- ✓ Con precedencia (* antes que +)
- ✓ Asociatividad

La gramática correcta fuerza:

$$\begin{aligned} &3 * 5 + 4 \\ &= (3 * 5) + 4 \end{aligned}$$

No:

$$3 * (5 + 4)$$

Por eso se usan gramáticas con E, T, F.

Recurso: <https://github.com/fool2fish/dragon-book-exercise-answers>



Resumen ultra corto (para memorizar)

- SDD = Gramática + Atributos + Reglas Semánticas
- Atributos:
 - Sintetizados: hijos → padre
 - Heredados: padre/hermanos → nodo
- Árbol anotado = árbol + valores
- Reglas semánticas = qué calcular
- Acciones semánticas = cómo implementarlo
- Se usa para:
 - Evaluar expresiones
 - Tipos
 - Generar código
 - Tabla de símbolos

LABORATORIO

USAC — Escuela de Ciencias y Sistemas

Organización de Lenguajes y Compiladores 2

Guía de Repaso — ANTLR4 + PHP

Análisis Léxico, Sintáctico, Semántico, DDS, Entornos, Funciones y Closures

1. Repaso General — Fases del Compilador

1.1 ¿Por qué estudiar compiladores?

Más allá de construir un compilador, el curso enseña habilidades fundamentales de ingeniería:

- Pensar en etapas: no todo se resuelve de una vez.
- Diseñar interfaces claras: cada fase recibe una entrada bien definida y produce una salida precisa.
- Aceptar la formalidad: las reglas permiten razonamiento automático.
- Separar preocupaciones (Separation of Concerns): una de las habilidades más importantes de la ingeniería moderna.

1.2 Las tres fases principales

Cada fase del compilador aporta una lección de diseño:

Fase	Qué hace	Lección clave
Análisis Léxico	Convierte texto en tokens	<i>No todo detalle es importante al mismo nivel; primero hay que delimitar el problema</i>
Análisis Sintáctico	Verifica estructura gramatical y construye árbol sintáctico	<i>Estructura antes que significado; pensar en reglas formales</i>
Análisis Semántico	Verifica tipos, alcance y significado	<i>Algo puede estar bien formado pero ser incorrecto</i>

1.3 Parseo LR vs LL — Resumen rápido

Aspecto	LL (Top-Down)	LR (Bottom-Up)
Dirección	Del axioma a las hojas (derivación)	De los tokens hacia el axioma (reducción)
Gramáticas	LL(1) — sin recursión izquierda, factorizada	LR(0), SLR, LALR, LR(1) — más potente
Herramientas	ANTLR4 (genera parsers LL(*) adaptativos)	Bison/Yacc (genera parsers LALR)

2. Definición Dirigida por la Sintaxis (DDS)

2.1 ¿Qué es una DDS?

Una Definición Dirigida por la Sintaxis es una gramática libre de contexto enriquecida con:

- Atributos: se asocian con los símbolos gramaticales (terminales y no terminales).
- Reglas semánticas: se asocian con cada producción y calculan los valores de los atributos.

Ejemplo canónico

$S \rightarrow S1 + T$ con regla $S.val = S1.val + T.val$

2.2 Tipos de Atributos

Atributos Sintetizados

El valor de un no terminal A en el nodo N se calcula a partir de los atributos de los HIJOS de N y del propio N. La información fluye de abajo hacia arriba en el árbol.

Definición formal

Un atributo sintetizado para un no terminal A en un nodo N se define mediante una regla semántica asociada con la producción en N, cuyo encabezado es A. Se define solo en términos de los atributos de los hijos de N y del mismo N.

Atributos Heredados

El valor de un no terminal B en el nodo N se calcula a partir de los atributos del PADRE de N, del mismo N, o de sus HERMANOS. La información fluye de arriba hacia abajo.

Definición formal

Un atributo heredado para un no terminal B en el nodo N se define mediante una regla semántica asociada con la producción en el padre de N. Se define solo en términos de los atributos del padre de N, del mismo N y de sus hermanos.

Ejemplo: Dado $T1 * T2 * T3$, la raíz del subárbol para $* T2$ hereda el valor de $T1$. La raíz del subárbol para $* T3$ hereda el valor de $T1 * T2$.

2.3 Árbol de Análisis Sintáctico Anotado

Un árbol sintáctico anotado (annotated parse tree) muestra, en cada nodo, el valor de sus atributos tras aplicar todas las reglas semánticas. Es la forma visual de una DDS evaluada sobre una frase concreta.

2.4 Grafos de Dependencias

Describe el flujo de información entre instancias de atributos en un árbol de análisis. Una flecha de atributo A a atributo B significa que el valor de A se necesita para calcular B.

- Las líneas punteadas representan el CST (árbol concreto de sintaxis).
- Las líneas sólidas representan el árbol anotado con los atributos.
- No importa el tipo de atributo; las flechas se dibujan en términos de la secuencia para crear el valor.

2.5 Falsos Sintetizados (Efectos Laterales)

Las reglas semánticas que se ejecutan por sus efectos adicionales (como `print(E.val)`) se tratan como definiciones de atributos falsos sintetizados asociados con el encabezado de la producción. Esto permite modelar acciones secundarias dentro del marco DDS.

2.6 Esquemas de Traducción Orientados por la Sintaxis

Son una notación complementaria a las DDS donde se insertan acciones semánticas directamente dentro del cuerpo de las producciones:

- Las acciones son fragmentos de programa incrustados y pueden aparecer en cualquier posición dentro del cuerpo de una producción.
- Toda DDS puede implementarse con esta notación.
- Una acción en el cuerpo se ejecuta tan pronto como los símbolos gramaticales a la izquierda de la acción tengan coincidencias.

DDS (Gramáticas Atribuidas)	Esquemas de Traducción
Sin efectos adicionales, cualquier orden de evaluación consistente con el grafo de dependencias	Imponen evaluación izquierda a derecha; las acciones pueden contener cualquier fragmento de programa

3. Tabla de Símbolos y Manejo de Entornos

3.1 Tabla de Símbolos

Estructura de datos clave que almacena información detallada sobre los identificadores del código fuente:

- Tipo del identificador (int, float, string, función, etc.)
- Ámbito (scope) donde fue declarado
- Dirección de memoria (real o lógica)
- Valor actual (si ya fue asignado)

Permite al compilador/intérprete realizar análisis semánticos, detectar errores y generar código eficiente.

3.2 Cadena de Entornos — Parent Pointer

En los lenguajes modernos, los scopes son anidados. La técnica del Parent Pointer modela esto con una cadena de entornos:

- Cada entorno (scope) contiene su tabla de símbolos local.
- Cada entorno tiene un puntero al entorno padre (scope que lo contiene).
- La búsqueda de un símbolo sube por la cadena hasta encontrarlo o llegar al entorno global.
- Casos de uso: bloques anidados, funciones dentro de módulos, lambdas, closures.

Analogía práctica

Imagina que buscas tus llaves: primero en tu habitación, luego en la sala, luego en el carro. El Parent Pointer funciona igual: busca en el entorno local, luego sube al padre, y así hasta el global.

3.3 Declaraciones vs Asignaciones

Declaración	Asignación
Introduce un nuevo símbolo en el entorno actual. Se registra en la tabla de símbolos. Se reserva espacio (real o lógico). Aún no tiene valor (o tiene valor por defecto).	Modifica un símbolo ya existente. Se busca en el entorno actual y luego en los padres. Se evalúa la expresión del lado derecho. Se guarda el nuevo valor en la entrada encontrada.

4. Marcadores, Atributos Heredados y Sistema de Tipos

4.1 Marcadores (Markers)

Llamamos marcadores a ciertas acciones semánticas o símbolos especiales que se introducen en la gramática para controlar el momento en que se computan algunos atributos o se realiza cierta acción. En parsers ascendentes (LR), las acciones se ejecutan al reducir. Esto es perfecto para atributos sintetizados, pero complica los heredados (info que debe "bajar" desde el padre o de izquierda a derecha).

Tipo 1 — Acciones intermedias (mid-rule actions)

Se escriben como { ... } dentro de la regla de Bison/Yacc. Son tratadas como si fueran un símbolo extra con su propio \$\$, que puedes usar después como \$k.

- Ventaja: pueden leer los valores de los símbolos ya vistos (\$1, \$2, ...).
- Uso típico: preparar contexto para lo que viene a continuación.

Tipo 2 — No terminal vacío M (marker)

Se declara como: $M : /* \text{vacío} */ \{ \dots \}$ y luego se incluye en la regla: $A : X M Y$

- Ventaja: fuerza un punto de reducción justo antes de Y.
- Útil para ejecutar código en un momento preciso del análisis ascendente.

4.2 Atributos Heredados en Detalle

Un atributo heredado es un valor que un no terminal recibe desde su padre o desde su hermano izquierdo en el árbol (baja información de contexto al subárbol).

Se usan para pasar:

- El tipo actual en una declaración (int, float, etc.)
- El nivel de indentación en un pretty-printer
- Un acumulador que se arrastra (suma parcial)
- El ámbito o información sobre el contexto ("estamos dentro de un bucle")

Ejemplo clásico

Evaluar: $\text{real id1, id2, id3D} \rightarrow T \text{ LT} \rightarrow \text{int} \mid \text{realL} \rightarrow L, \text{id} \mid \text{id}$ El tipo 'real' declarado en T debe heredarse a L para que cada identificador sepa su tipo.

Las gramáticas L-atribuidas permiten evaluar todos los atributos (tanto sintetizados como heredados compatibles) en una sola pasada de izquierda a derecha, haciéndolas ideales para parsers LL y ANTLR.

4.3 Sistema y Verificación de Tipos

Un sistema de tipos define los tipos de datos del lenguaje y las reglas de cómo se pueden usar e interactuar entre sí.

La verificación de tipos (type checking) es el proceso de comprobar que cada operación reciba operandos del tipo apropiado según las reglas del lenguaje.

Fuertemente tipado vs Débilmente tipado

Fuertemente Tipado (Strong)	Débilmente Tipado (Weak)
Prohíbe operaciones entre tipos incompatibles. Requiere conversiones explícitas. Ej: C#, Java	Permite mezclas con conversiones implícitas (coerción). Ej: JavaScript, Perl, PHP
C#: $"1" + 1 \rightarrow \text{Error de compilación}$	Perl: $"1" + 1 \rightarrow 2$ (convierte automáticamente)

Principios SOLID en el Intérprete

Para implementar un sistema de tipos limpio, se aplican principios SOLID:

- Principio de Responsabilidad Única (SRP): la acción semántica no debe guardar la operación directamente.
- Principio Abierto/Cerrado (OCP): cada tipo de expresión tiene su propia clase con método `interpret()`.
- La acción semántica funciona como fábrica abstracta (Abstract Factory).

```
// Malo - viola SRP y OCP
expr: expr '+' expr { $$ = guardarOperacion('+', $1, $3); }
```

```
// Bueno - cada expresion tiene su clase
expr: expr '+' expr { $$ = nuevoSumaExpresion($1, $3); }
    | expr '-' expr { $$ = nuevoRestaExpresion($1, $3); }
```


5. Descenso Recursivo, Funciones y Closures

5.1 Análisis Sintáctico de Descenso Recursivo

Un analizador de descenso recursivo es un método top-down donde cada no terminal de la gramática se implementa como una función en el código del compilador/intérprete. Estas funciones se llaman recursivamente procesando la entrada según las producciones.

- Corresponde a gramáticas LL(1) típicamente.
- La estructura del código sigue directamente la estructura de la gramática.
- Si la gramática tiene $A \rightarrow X Y Z$, la función $A()$ llama sucesivamente a $X()$, $Y()$ y $Z()$.
- Requiere que la gramática NO tenga recursión izquierda inmediata y esté factorizada.

Traducción durante el análisis

Se amplían las funciones del parser para que, además de verificar la sintaxis, realicen acciones semánticas: generar código, poblar la tabla de símbolos, calcular valores, etc. Cada función puede recibir atributos heredados como parámetros y retornar atributos sintetizados como valor de retorno.

Si la gramática es L-atribuida, el análisis descendente puede garantizar que en el momento de ejecutar una acción semántica, todos los datos necesarios ya están disponibles (traducción en una sola pasada).

5.2 Diagramas de Transición para Parsers LL

Un diagrama de transición es una representación gráfica de una gramática LL (predictiva) donde cada no terminal es un pequeño autómata con:

- Un estado de entrada y un estado de aceptación.
- Arcos etiquetados por terminales (consumen ese símbolo), no terminales (llaman al subdiagrama) o ϵ (transición vacía).

Es equivalente a una tabla LL(1) pero más visual. Cada producción $A \rightarrow X_1 X_2 \dots X_k$ se convierte en un camino con arcos por $X_1, X_2, \dots, X_k$. Las alternativas ramifican desde el estado de entrada.

Ejemplo — Gramática de expresiones LL(1):

```
E → T E'
E' → '+' T E' | '-' T E' | ε
T → F T'
T' → '*' F T' | ε
F → '(' E ')' | id
```

5.3 Funciones Nativas (Embebidas)

Las funciones nativas (built-in) son fundamentales porque proporcionan acceso directo a servicios del sistema sin que el desarrollador deba implementarlos desde cero (manejo de archivos, E/S, conversión de tipos, etc.).

Se registran en la tabla de símbolos del entorno global al iniciar el intérprete, antes de ejecutar cualquier código del usuario.

5.4 Funciones Foráneas (User-Defined)

Las funciones definidas por el usuario encapsulan instrucciones bajo un identificador para facilitar la reutilización y organización del código.

- Se almacenan en la tabla de símbolos del entorno donde fueron declaradas.
- Pueden invocarse como cualquier otra función del lenguaje una vez registradas.

- Tienen su propio entorno de ejecución (scope local) con Parent Pointer al entorno de definición.

5.5 Closures (Clausuras)

Una clausura (closure) es el mecanismo que permite que una función conserve acceso a las variables de su entorno de definición, incluso cuando es invocada en un contexto diferente.

La función mantiene una referencia al entorno en que fue creada (no al entorno en que es llamada). Esto le permite usar valores declarados fuera de su ámbito inmediato de ejecución.

Clave conceptual

Closure = Función + Entorno de definición capturado. Al invocar la función, primero busca en su entorno local, luego en el entorno que capturó (closure), y así sube por la cadena de Parent Pointers.

5.6 Resolución de Variables y Hoisting

Resolución de Variables

Cuando se accede a una variable dentro de una función, el intérprete sigue la cadena de entornos: primero el local, luego el closure capturado, luego el global.

Hoisting

El hoisting permite que ciertas declaraciones sean registradas en el entorno antes de la ejecución, haciendo posible usar identificadores antes de que aparezcan explícitamente en el código fuente.

- En JavaScript: declaraciones var y function se "elevan" al inicio del scope.
- En un intérprete propio: se puede implementar con dos pasadas (una de registro, una de ejecución).

5.7 Call Stack (Pila de Llamadas)

El Call Stack es la estructura que gestiona el estado de ejecución de cada función activa:

- Cada invocación de función crea un nuevo marco (stack frame) con su entorno local.
- Al retornar, el marco se destruye y se vuelve al marco anterior.
- Permite la recursión porque cada llamada tiene su propio estado aislado.
- El stack overflow ocurre cuando la pila crece sin límite (recursión infinita).

6. Parte Práctica — ANTLR4 + PHP

6.1 ¿Qué es ANTLR4?

ANTLR4 (ANother Tool for Language Recognition) es un generador de parsers que a partir de una gramática .g4 genera automáticamente el Lexer, el Parser y las interfaces para el árbol sintáctico.

- Genera parsers LL(*) adaptativos — más potentes que LL(1).
- Soporta múltiples lenguajes objetivo: Java, Python, C#, PHP, JavaScript, etc.
- Genera el árbol CST (Concrete Syntax Tree) automáticamente.
- Permite recorrer el árbol con los patrones Listener (Observer) o Visitor.

6.2 Estructura de una Gramática .g4

```
grammar NombreGramatica;

// Reglas del parser (empiezan con minúscula)
programa : sentencia+ EOF ;
sentencia : expr SEMICOLON
          | declaracion
          ;
expr : expr OP_MAS expr      # exprSuma
     | expr OP_MENOS expr    # exprResta
     | expr OP_MUL expr      # exprMult
     | expr OP_DIV expr      # exprDiv
     | LPAREN expr RPAREN    # exprAgrupacion
     | NUMERO                # exprNumero
     ;

// Reglas del lexer (empiezan con MAYUSCULA)
NUMERO : [0-9]+ ('.' [0-9]+)? ;
OP_MAS : '+' ;
OP_MENOS : '-' ;
OP_MUL : '*' ;
OP_DIV : '/' ;
LPAREN : '(' ;
RPAREN : ')' ;
SEMICOLON: ';' ;
WS : [ \t\r\n]+ -> skip ;
```

6.3 Patrón Visitor en PHP

El patrón Visitor es el más recomendado para intérpretes porque:

- Separa la lógica de recorrido del árbol de la lógica semántica.
- Cumple el Principio Abierto/Cerrado: nuevas operaciones sin modificar el árbol.
- Cada tipo de nodo tiene su propio método visit, manteniendo el Principio de Responsabilidad Única.

Estructura del Visitor en PHP:

```
<?php
require_once 'vendor/autoload.php';
use Antlr\Antlr4\Runtime\Tree\ParseTreeVisitor;

class Interpreter extends NombreGramaticaBaseVisitor {
```

```

public function visitExprSuma($ctx) {
    $left = $this->visit($ctx->expr(0));
    $right = $this->visit($ctx->expr(1));
    return $left + $right;
}

public function visitExprResta($ctx) {
    $left = $this->visit($ctx->expr(0));
    $right = $this->visit($ctx->expr(1));
    return $left - $right;
}

public function visitExprMult($ctx) {
    $left = $this->visit($ctx->expr(0));
    $right = $this->visit($ctx->expr(1));
    return $left * $right;
}

public function visitExprDiv($ctx) {
    $left = $this->visit($ctx->expr(0));
    $right = $this->visit($ctx->expr(1));
    if ($right == 0) throw new \Exception('Division por cero');
    return $left / $right;
}

public function visitExprNumero($ctx) {
    return (float) $ctx->NUMERO()->getText();
}

public function visitExprAgrupacion($ctx) {
    return $this->visit($ctx->expr());
}
}

```

6.4 Entorno (Environment) en PHP

Implementación básica de la tabla de símbolos con Parent Pointer:

```

<?php
class Environment {
    private array $values = [];
    private ?Environment $parent;

    public function __construct(?Environment $parent = null) {
        $this->parent = $parent;
    }

    // Declarar nueva variable en este scope
    public function declare(string $name, mixed $value): void {
        $this->values[$name] = $value;
    }

    // Asignar - busca en cadena de entornos
    public function assign(string $name, mixed $value): void {
        if (array_key_exists($name, $this->values)) {

```

```

        $this->values[$name] = $value;
        return;
    }
    if ($this->parent !== null) {
        $this->parent->assign($name, $value);
        return;
    }
    throw new \Exception("Variable '$name' no declarada");
}

// Obtener - busca en cadena de entornos
public function get(string $name): mixed {
    if (array_key_exists($name, $this->values)) {
        return $this->values[$name];
    }
    if ($this->parent !== null) {
        return $this->parent->get($name);
    }
    throw new \Exception("Variable '$name' no definida");
}
}

```

6.5 Closure en PHP

```

class FuncionUsuario {
    public array      $params;
    public mixed      $cuerpo;    // nodo del AST
    public Environment $closure;  // entorno capturado

    public function __construct(
        array $params,
        mixed $cuerpo,
        Environment $closure
    ) {
        $this->params = $params;
        $this->cuerpo = $cuerpo;
        $this->closure = $closure; // snapshot del entorno actual
    }
}

// Al invocar la funcion:
public function invocar(FuncionUsuario $fn, array $args): mixed {
    // Nuevo entorno hijo del closure (no del entorno actual)
    $entornoLocal = new Environment($fn->closure);
    foreach ($fn->params as $i => $param) {
        $entornoLocal->declare($param, $args[$i]);
    }
    return $this->ejecutar($fn->cuerpo, $entornoLocal);
}

```

6.6 Sistema de Tipos básico en PHP

Para el verificador de tipos, se puede implementar una matriz de tipos compatible:

```

// Matriz de compatibilidad de tipos para operaciones aritméticas
$tiposCompatibles = [

```

```

        'int'    => ['int' => 'int',    'float' => 'float', 'string' => 'error'],
        'float' => ['int' => 'float',    'float' => 'float', 'string' => 'error'],
        'string' => ['int' => 'error',    'float' => 'error', 'string' => 'string'],
    ];

function verificarTipos(string $tipoIzq, string $tipoDer, string $op): string {
    global $tiposCompatibles;
    if ($op === '+' && $tipoIzq === 'string' && $tipoDer === 'string') {
        return 'string'; // concatenacion
    }
    return $tiposCompatibles[$tipoIzq][$tipoDer] ?? 'error';
}

```

6.7 Flujo Completo — Punto de Entrada

```

<?php
// index.php - punto de entrada del interprete
require_once 'vendor/autoload.php';

$input      = \Antlr\Antlr4\Runtime\ANTLRInputStream::fromPath('programa.txt');
$lexer      = new NombreGramaticaLexer($input);
$tokens     = new \Antlr\Antlr4\Runtime\CommonTokenStream($lexer);
$parser     = new NombreGramaticaParser($tokens);

$tree       = $parser->programa(); // regla inicial
$interpreter = new Interpreter();
$resultado  = $interpreter->visit($tree);

echo "Resultado: " . $resultado . PHP_EOL;

```

7. Referencia Rápida — Resumen de Conceptos Clave

Término	Definición rápida
DDS	Gramática libre de contexto + atributos + reglas semánticas por producción.
Atributo sintetizado	Se calcula de abajo hacia arriba (desde los hijos).
Atributo heredado	Se calcula de arriba hacia abajo (desde el padre/hermanos).
Árbol anotado	Árbol sintáctico con los valores de los atributos en cada nodo.
Grafo de dependencias	Muestra qué atributo necesita a cuál para calcularse.
Marcador (Marker)	Símbolo vacío M o acción mid-rule que fuerza un punto de ejecución en parsers LR.
Tabla de símbolos	Estructura que mapea identificadores a su tipo, scope y valor.
Parent Pointer	Puntero de un entorno a su entorno padre para modelar scopes anidados.
Closure	Función + referencia al entorno en que fue definida (no en que se llama).
Hoisting	Registrar declaraciones antes de ejecutar para usarlas antes de su definición.
Call Stack	Pila de marcos de activación; cada llamada a función crea uno nuevo.
LL(1)	Parser predictivo top-down con 1 token de lookahead; sin recursión izquierda.
LR	Parser bottom-up shift-reduce; más potente, soporta más gramáticas.
ANTLR4	Generador de parsers LL(*) a partir de gramáticas .g4.
Visitor (ANTLR4)	Patrón para recorrer el AST; visitXxx() por cada tipo de regla etiquetada.
L-atribuida	Gramática donde los atributos heredados solo dependen del padre o hermanos izquierdos.
Tipo fuerte/débil	Fuerte: requiere conversiones explícitas. Débil: hace coerción implícita.

OLC2 — USAC Sección B | Material de repaso pre-práctica ANTLR4 + PHP

Requisitos:

- Usaremos linux en el transcurso del curso.
- Instalaremos las siguientes herramientas:

```
pip install antlr4-tools
sudo apt install php composer
```

- Además de configurar lo siguiente:

```
composer require antlr/antlr4-php-runtime
```

- Debemos definir nuestra grammar en un archivo, en este caso `Grammar.g4`.
- Ejecutaremos el siguiente comando para generar el parser y la interfaz del visitante:

```
antlr4 -Dlanguage=PHP Grammar.g4 -visitor
```

Podcast: Luis Furlán

Resumen del Podcast: Luis Furlán

El Padre de Internet en Guatemala

La historia de la conectividad en Guatemala tiene un protagonista indiscutible: el ingeniero Luis Roberto Furlán. A inicios de la década de 1990, cuando las únicas opciones de comunicación eran el teléfono, el correo postal, el fax y el telegrama, Furlán buscaba una forma más eficiente de mantenerse en contacto con colegas e investigadores para intercambiar grandes volúmenes de datos. Fue en 1990 cuando escuchó por primera vez sobre Internet, y aunque conectarse requería computadoras del tamaño de una habitación, encontró una solución alternativa: utilizando una PC común y un nodo UUCP, protocolo que convertía información digital en señal analógica para transmitirla por línea telefónica, logró establecer la primera conexión a Internet en Guatemala.

El siguiente paso natural fue otorgar una identidad digital propia al nodo universitario. Furlán, solicitó inicialmente el dominio `uvg.edu.gt` ante la Universidad del Sur de California, entidad responsable de la asignación de nombres en Internet en esa época. Sin embargo, al ser la UVG la primera institución guatemalteca conectada, se les ofreció adoptar el dominio del país, dando origen al `.gt`. En 1992, John Postel, uno de los pioneros del Internet en Estados Unidos, encomendó a Furlán la administración del dominio `.gt`, con la responsabilidad de evitar nombres duplicados y garantizar una distribución imparcial.

Furlán no solo fue pionero en la infraestructura tecnológica del país, sino también en la integración de la tecnología con la docencia universitaria. Como director del Centro de Estudios en Informática Aplicada (CEIA) de la UVG y jefe del Departamento de Ciencia de la Computación, impulsó la convicción de que la solución a los problemas del país pasa necesariamente por la tecnología. Su filosofía educativa se tradujo en la creación de programas académicos orientados a formar profesionales con sólidas competencias técnicas y visión innovadora. Furlán promovió activamente la aplicación de la tecnología en el aula, no solo como herramienta pedagógica sino como objeto de estudio y campo de investigación, sentando las bases de lo que hoy es una tradición de excelencia en ciencias de la computación en Guatemala.

En el podcast, Furlán aborda los fundamentos que sostienen las ciencias de la computación como disciplina. Estos pilares incluyen la algoritmia y las estructuras de datos, la teoría de la computación, la ingeniería de software, los sistemas operativos y redes, y más recientemente, la inteligencia artificial y la ciencia de datos. Para Furlán, las ciencias de la computación no son simplemente una técnica, sino una forma de pensamiento que permite resolver problemas complejos en cualquier ámbito. La rigurosidad matemática y lógica es la columna vertebral de todo lo demás, y es desde ese fundamento que se construyen todas las aplicaciones tecnológicas modernas.

Uno de los campos que Furlán destaca con especial entusiasmo es la ciencia de datos. A través de técnicas estadísticas, aprendizaje automático y análisis computacional, esta disciplina permite extraer conocimiento valioso a partir de grandes volúmenes de información. Los resultados que pueden obtenerse son variados y de alto impacto: desde modelos predictivos para decisiones empresariales y políticas públicas, hasta herramientas de diagnóstico médico, análisis de fenómenos sociales y ambientales, detección de fraudes y personalización de servicios. En el contexto guatemalteco, la ciencia de datos representa una oportunidad enorme para abordar desafíos nacionales concretos, como la pobreza, el cambio climático y la desigualdad, siempre que se cuente con profesionales capacitados y datos confiables.

Un hito emblemático de la investigación tecnológica guatemalteca es el satélite Quetzal-1, el primer nanosatélite diseñado y ensamblado en Guatemala, desarrollado como proyecto académico y de investigación desde 2014. El Quetzal-1 demostró que Guatemala tiene la capacidad humana e institucional de participar en proyectos de alta tecnología a nivel internacional.

La inteligencia artificial ha irrumpido con fuerza en el ámbito universitario, y Furlán analiza esta relación con claridad. Por un lado, la IA representa una herramienta poderosa para la investigación, el diseño de nuevos cursos y la personalización del aprendizaje. Por otro lado, genera interrogantes serias sobre la evaluación académica y la integridad intelectual. En la UVG, la IA no se percibe como una amenaza, sino como un campo de estudio fundamental y una herramienta que debe integrarse con criterio. La universidad ha apostado por formar a sus estudiantes

tanto en el uso de estas tecnologías como en la comprensión de sus fundamentos matemáticos y algorítmicos, de modo que sean capaces de innovar y no solo de consumir herramientas externas.

El uso de herramientas de inteligencia artificial generativa, como los asistentes de código basados en modelos de lenguaje, plantea un desafío particular en la enseñanza de la programación. Furlán plantea que la clave no es prohibir estas herramientas, sino rediseñar la forma en que se evalúa a los estudiantes, priorizando la comprensión profunda, la resolución de problemas originales y el pensamiento crítico sobre la mera producción de código. Las evaluaciones orales, los proyectos contextualizados y el análisis de casos reales son algunas de las estrategias que permiten verificar el aprendizaje genuino. El objetivo es que el estudiante pueda entender lo que produce la IA, corregirlo y adaptarlo, lo cual requiere una base sólida en fundamentos de computación.

El trabajo en políticas públicas vinculadas a la tecnología es, según Furlán, un campo en el que Guatemala todavía tiene mucho camino por recorrer. A lo largo de su trayectoria, ha participado activamente en iniciativas de gobernanza de Internet y en la construcción de redes académicas nacionales como Mayanet y la Red Avanzada Guatemalteca de Investigación y Educación. Su conclusión es que las políticas públicas en materia tecnológica deben ser construidas con la participación activa de la academia, el sector privado y la sociedad civil, evitando que queden relegadas a decisiones puramente burocráticas o comerciales. Asimismo, subraya que el Estado guatemalteco debe reconocer la tecnología como un factor estratégico de desarrollo y destinar recursos sostenidos a la investigación, la infraestructura y la formación de talento humano especializado.

Para cerrar el podcast, Furlán comparte un consejo que refleja toda su trayectoria de vida: la curiosidad y el aprendizaje continuo son las virtudes más valiosas para cualquier estudiante o profesional en el campo tecnológico. En un entorno donde las herramientas y los paradigmas cambian velozmente, lo que realmente marca la diferencia es la capacidad de adaptarse, de aprender de manera autónoma y de no tener miedo a enfrentar lo desconocido.

Proyecto 1

GOLAMPI INTERPRETER

Plan de Proyecto — OLC2 Sección B

Universidad San Carlos de Guatemala | Ingeniería en Ciencias y Sistemas

Campo	Detalle
Ponderación	45 puntos
Tiempo estimado	40 horas
Fecha de entrega	12/03/2026
Calificación	13/03/2026 — 28/03/2026
Sección	B — Auxiliar: rubenralda
Stack	ANTLR4 + PHP + HTML/CSS/JS

1. Resumen Ejecutivo del Proyecto

El proyecto Golampi Interpreter consiste en el desarrollo de un intérprete funcional para un lenguaje académico llamado Golampi, cuya sintaxis y semántica están inspiradas en Go (Golang). El objetivo central es aplicar de forma práctica los conceptos teóricos de compiladores: análisis léxico, sintáctico y semántico, implementados sobre las herramientas ANTLRv4 y PHP.

1.1 ¿Qué es Golampi?

Golampi es un lenguaje de programación estáticamente tipado, diseñado con fines académicos. Comparte la estructura sintáctica de Go pero con un subconjunto reducido de características, orientado a demostrar el funcionamiento de un intérprete completo. Sus características principales son:

- Tipos estáticos: int32, float32, bool, rune, string.
- Variables con declaración explícita (var) y declaración corta (:=).
- Constantes inmutables (const).
- Estructuras de control: if/else, switch, for (cubre while e infinito).
- Sentencias de transferencia: break, continue, return.
- Arreglos unidimensionales y multidimensionales de tamaño fijo.
- Funciones con parámetros por valor y por referencia (punteros).
- Múltiples valores de retorno por función.
- Hoisting de funciones: se pueden llamar antes de su definición textual.
- Funciones embebidas: fmt.Println, len, now, substr, typeOf.
- Punto de entrada único: función main() sin parámetros ni retorno.
- Comentarios de una línea (//) y multilínea (/* ... */).
- Evaluación de cortocircuito en operadores lógicos (&& y ||).

1.2 Arquitectura del Sistema

La solución sigue una arquitectura monolítica cliente/servidor. El backend en PHP ejecuta toda la lógica del intérprete (generada a partir de la gramática ANTLR4), y el frontend en HTML/CSS/JS actúa únicamente como interfaz de usuario. La comunicación entre ambas capas se realiza mediante solicitudes HTTP.

El flujo de ejecución es el siguiente:

- El usuario escribe o carga código Golampi en el editor web.
- Al presionar Ejecutar, el frontend envía el código fuente al servidor PHP.
- El backend tokeniza el código (análisis léxico) con el lexer generado por ANTLR4.
- El parser (también generado por ANTLR4) construye el Árbol Sintáctico Concreto (CST).
- El Visitor PHP recorre el CST realizando análisis semántico y ejecución simultáneos.
- El resultado (salida de consola, errores y tabla de símbolos) se devuelve al frontend.
- El frontend muestra los resultados y habilita la descarga de reportes.

1.3 Entregables del Proyecto

Entregable	Descripción
Prototipo funcional	Versión ejecutable del intérprete con las funcionalidades esenciales.
Gramática ANTLR4 (.g4)	Archivo de gramática formal del lenguaje Golampi.
Código PHP (Visitor)	Implementación del intérprete con manejo de entornos y tabla de símbolos.
Interfaz gráfica (GUI)	Editor web con consola, barra de acciones y panel de reportes.
Documentación técnica	Markdown con gramática formal, diagrama de clases y flujo de tabla de símbolos.
Manual de usuario	Guía de instalación, uso y descripción de reportes.
Repositorio GitHub	Código fuente organizado; auxiliar rubenralda agregado como colaborador.

2. Arquitectura de Carpetas y Organización del Proyecto

La siguiente estructura de carpetas refleja la separación entre frontend, backend, gramática y documentación. Está diseñada para ser clara, mantenible y alineada con la arquitectura monolítica definida en el enunciado.

2.1 Estructura General

```
golampi-interpreter/
├── grammar/                                # Archivos ANTLR4
│   └── Golampi.g4                          # Gramática del lenguaje
├── backend/                                # Lógica del intérprete en PHP
│   ├── generated/                          # Código PHP generado por ANTLR4
│   │   ├── GolampiLexer.php
│   │   ├── GolampiParser.php
│   │   ├── GolampiBaseVisitor.php
│   │   └── GolampiVisitor.php
│   ├── interpreter/                        # Implementación del intérprete
│   │   ├── Interpreter.php                # Visitor principal
│   │   ├── Environment.php                # Entornos y tabla de símbolos
│   │   ├── FuncionUsuario.php             # Representación de funciones (closure)
│   │   └── ErrorHandler.php               # Recolección de errores
│   ├── models/                            # Clases de datos
│   │   ├── Symbol.php                    # Símbolo (id, tipo, valor, ámbito, línea)
│   │   └── ErrorEntry.php                 # Entrada de error (tipo, desc, línea, col)
│   ├── reports/                           # Generadores de reportes
│   │   ├── SymbolTableReport.php
│   │   └── ErrorReport.php
│   └── api/                               # Endpoints HTTP
│       └── execute.php                    # Punto de entrada: recibe código, retorna
JSON
├── frontend/                              # Interfaz de usuario
│   ├── index.html                        # Página principal
│   ├── css/
│   │   └── style.css
│   └── js/
│       ├── editor.js                     # Lógica del editor de código
│       ├── api.js                         # Comunicación con el backend
│       └── reports.js                     # Descarga de reportes
```



```

├── docs/                                # Documentación técnica
│   ├── README.md
│   ├── gramatica.md                    # Gramática formal documentada
│   ├── diagrama-clases.md             # Diagrama de clases
│   └── manual-usuario.md               # Manual de instalación y uso
├── tests/                              # Casos de prueba Golampi
│   ├── aritmetica.gol
│   ├── control-flujo.gol
│   ├── arreglos.gol
│   ├── funciones.gol
│   └── errores.gol
└── composer.json / .htaccess

```

2.2 Descripción de Componentes Clave

Archivo / Carpeta	Responsabilidad
Golampi.g4	Define la gramática formal: reglas léxicas y sintácticas del lenguaje.
generated/	Código PHP auto-generado por ANTLR4. No se edita manualmente.
Interpreter.php	Visitor principal: recorre el CST, evalúa expresiones y ejecuta sentencias.
Environment.php	Implementa entornos anidados con parent pointer. Maneja declare/assign/get.
FuncionUsuario.php	Representa una función definida por el usuario: params, cuerpo y closure (entorno de definición).
ErrorHandler.php	Recolecta errores léxicos, sintácticos y semánticos sin detener la ejecución.
execute.php	Endpoint PHP que recibe el código fuente, instancia el intérprete y devuelve JSON con salida, errores y tabla de símbolos.
editor.js	Maneja el textarea del editor, carga/guardado de archivos y envío al backend.
api.js	Realiza el fetch POST a execute.php y distribuye la respuesta a consola y reportes.
tests/	Archivos .gol con programas Golampi de prueba para validar cada funcionalidad.

2.3 Flujo de Datos entre Capas

El flujo completo desde que el usuario presiona Ejecutar hasta que ve el resultado:

Paso	Capa	Acción
------	------	--------

1	Frontend (JS)	Captura el texto del editor y hace POST /api/execute.php con { codigo: '...' }
2	Backend (PHP)	execute.php recibe el JSON, instancia el Lexer y Parser de ANTLR4
3	Backend (ANTLR4)	El Lexer tokeniza; el Parser construye el CST
4	Backend (Interpreter)	El Visitor recorre el CST: evalúa, asigna, ejecuta, captura errores
5	Backend (Reports)	Se generan los objetos de tabla de símbolos y lista de errores
6	Backend (PHP)	execute.php serializa todo a JSON: { output, errors, symbols }
7	Frontend (JS)	api.js muestra output en consola, habilita descarga de reportes

3. Plan de Sprints

El proyecto se divide en 5 sprints de desarrollo, organizados de forma gradual para construir el intérprete de manera incremental. Cada sprint produce un entregable funcional y verificable, reduciendo el riesgo de deuda técnica acumulada.

Sprint	Nombre	Duración	Fechas
1	Cimientos: Gramática + Entorno	3 días	02/03 — 04/03
2	Expresiones y Variables	2 días	05/03 — 06/03
3	Control de Flujo y Arreglos	2 días	07/03 — 08/03
4	Funciones, Punteros y Builtins	2 días	09/03 — 10/03
5	GUI, Reportes e Integración Final	2 días	11/03 — 12/03

Sprint 1: Cimientos: Gramática + Entorno	
Fechas	02/03/2026 — 04/03/2026
Objetivo	Al finalizar este sprint, el pipeline ANTLR4 → PHP está funcionando end-to-end. Se puede parsear un programa vacío sin errores.
Tareas	• Instalar y configurar ANTLR4 + PHP runtime (antlr4-php-runtime). • Escribir el archivo Golampi.g4 con reglas léxicas básicas: literales, identificadores, comentarios, operadores. • Agregar reglas sintácticas del programa: programa, sentencia, bloque. • Generar código PHP desde la gramática con el comando antlr4. • Implementar la clase Environment.php con métodos declare(), assign(), get(). • Implementar Environment con parent pointer para scopes anidados. • Crear la clase Symbol.php y ErrorEntry.php. • Crear el esqueleto de Interpreter.php extendiendo GolampiBaseVisitor. • Implementar execute.php: recibir código, instanciar lexer/parser, retornar JSON básico. • Prueba de humo: enviar 'func main() {}' y recibir respuesta sin errores.

Sprint 2: Expresiones y Variables	
Fechas	05/03/2026 — 06/03/2026

Objetivo	<i>Al finalizar, el intérprete puede declarar variables de cualquier tipo, evaluar expresiones complejas e imprimir resultados. Es el núcleo funcional del lenguaje.</i>
Tareas	• Ampliar gramática: reglas para expresiones aritméticas (+, -, *, /, %), relacionales y lógicas. • Agregar reglas para declaración de variables (var) y declaración corta (:=). • Agregar reglas para constantes (const) y valor nil. • Implementar operadores de asignación compuesta (+=, -=, *=, /=). • Implementar visitVarDecl, visitShortDecl, visitConstDecl en Interpreter.php. • Implementar visitExprArith, visitExprRelational, visitExprLogical con chequeo de tipos. • Implementar las tablas de compatibilidad de tipos para cada operador. • Implementar evaluación de cortocircuito en && y . • Implementar fmt.Println como builtin básico para poder ver resultados. • Crear pruebas: variables con todos los tipos, operaciones aritméticas, impresión.

Sprint 3: Control de Flujo y Arreglos

Fechas	07/03/2026 — 08/03/2026
Objetivo	<i>Al finalizar, el intérprete soporta el control de flujo completo y arreglos multidimensionales. Se puede implementar bubble sort en Golampi.</i>
Tareas	• Agregar reglas gramaticales para if/else, switch/case/default. • Agregar reglas para los tres tipos de for (clásico, while, infinito). • Agregar reglas para break, continue y return. • Implementar visitIfStmt, visitSwitchStmt, visitForStmt en Interpreter.php. • Implementar break/continue con excepciones PHP o flags de control de flujo. • Agregar reglas gramaticales para arreglos: declaración simple, inicializada, multidimensional. • Implementar visitArrayDecl, visitArrayAccess, visitArrayAssign. • Implementar inicialización automática con valor por defecto según tipo. • Crear pruebas: if anidado, switch con múltiples cases, for clásico, bucle while, arreglo 2D con for anidado.

Sprint 4: Funciones, Punteros y Builtins

Fechas	09/03/2026 — 10/03/2026
---------------	-------------------------

Objetivo	<i>Al finalizar, el intérprete es completo funcionalmente. Se pueden escribir programas Golampi no triviales con funciones, closures y todas las builtins.</i>
Tareas	• Agregar reglas gramaticales para declaración de funciones (con y sin retorno, múltiples retornos). • Agregar reglas para parámetros por valor y por referencia (punteros *T y operadores & y *). • Implementar FuncionUsuario.php con closure al entorno de definición. • Implementar hoisting: registrar todas las funciones antes de ejecutar main. • Implementar visitFuncDecl y visitFuncCall en Interpreter.php. • Implementar paso por valor y paso por referencia con punteros. • Implementar múltiples valores de retorno. • Implementar las funciones embebidas restantes: len(), now(), substr(), typeOf(). • Implementar recolección completa de errores semánticos: variable no declarada, redeclaración, incompatibilidad de tipos. • Crear pruebas: funciones recursivas, paso por referencia con arreglos, múltiples retornos.

Sprint 5: GUI, Reportes e Integración Final	
Fechas	11/03/2026 — 12/03/2026
Objetivo	<i>Al finalizar, la aplicación es completamente funcional, tiene interfaz gráfica, reportes descargables y documentación completa lista para calificación.</i>
Tareas	• Desarrollar index.html con la estructura de la GUI: barra de acciones, editor, consola, panel de reportes. • Implementar style.css: diseño limpio y funcional, similar al mockup del enunciado. • Implementar editor.js: carga de archivos, guardado, limpieza de editor y consola. • Implementar api.js: POST a execute.php, manejo de respuesta JSON, mostrar output en consola. • Implementar reports.js: descarga de tabla de símbolos, errores y resultado como archivos. • Implementar SymbolTableReport.php: generar HTML/CSV con la tabla de símbolos. • Implementar ErrorReport.php: generar HTML/CSV con la tabla de errores. • Probar la aplicación end-to-end con todos los casos de prueba de los sprints anteriores. • Redactar documentación técnica en Markdown (gramática, diagrama de clases). • Redactar manual de usuario con capturas de pantalla. • Agregar colaborador rubenralda al repositorio de GitHub.

4. Referencia Rápida de Tipos y Operaciones

Resumen de las reglas de compatibilidad de tipos para implementar correctamente el chequeo semántico en el Visitor.

4.1 Tipos Primitivos

Tipo	Descripción	Valor por Defecto
int32	Entero con signo de 32 bits	0
float32	Punto flotante IEEE-754 de 32 bits	0.0
bool	Valor lógico true / false	false
rune	Carácter Unicode (alias de int32)	'\u0000'
string	Cadena de texto Unicode	""

4.2 Reglas de Promoción (Suma y Resta)

Operandos	Resultado
int32 + int32	int32
int32 + float32	float32
int32 + rune	int32
float32 + float32	float32
float32 + rune	float32
rune + rune	int32
string + string	string (solo suma)
int32 * string	string (repetición)
Cualquier + nil	nil (error)

4.3 Funciones Embebidas

Función	Parámetros	Retorno	Descripción
fmt.Println(...)	Variádico	void	Imprime valores separados por espacio + salto de línea
len(x)	string array	int32	Longitud de cadena o arreglo
now()	ninguno	string	Fecha y hora actual: YYYY-MM-DD HH:MM:SS
substr(s, i, n)	string, int, int	string	Subcadena desde índice i con longitud n

typeof(x)	cualquier tipo	string	Nombre del tipo como string: int32, float32, etc.
-----------	----------------	--------	---

5. Checklist de Calificación

Lista de verificación basada en los requisitos del enunciado para asegurar que nada quede pendiente antes de la entrega.

5.1 Funcionalidades Esenciales (Alcance Obligatorio)

#	Requisito	Sprint
1	Gramática ANTLR4 completa (.g4) generando lexer y parser PHP	1
2	Operaciones aritméticas: +, -, *, /, %	2
3	Operaciones relacionales y lógicas con cortocircuito	2
4	Declaración de variables: var, :=, const, nil	2
5	Todos los tipos: int32, float32, bool, rune, string	2
6	Sentencia if / else if / else	3
7	Sentencia switch / case / default (múltiples casos por case)	3
8	Bucle for: clásico, while, infinito	3
9	Sentencias break, continue, return	3
10	Arreglos unidimensionales y multidimensionales	3
11	Funciones con parámetros y retorno	4
12	Múltiples valores de retorno	4
13	Paso por referencia con punteros (& y *)	4
14	Hoisting de funciones	4
15	Función main como único punto de entrada	4
16	Builtins: fmt.Println, len, now, substr, typeOf	4
17	GUI con editor, consola y panel de reportes	5
18	Reporte de errores (léxicos, sintácticos, semánticos)	5
19	Reporte tabla de símbolos	5
20	Repositorio GitHub con auxiliar rubenralda agregado	5

Ejemplos y bloques del enunciado en TEXTO PLANO

1. Identificadores (reglas formales)

```
identifier = letter { letter | digit } .  
letter    = unicode_letter | "_" .  
digit     = unicode_digit .
```

2. Comentarios

```
// Este es un comentario de una sola línea
```

```
/*  
  Este es un comentario  
  de múltiples líneas  
*/
```

3. Ejemplo de Bloques y Scope

```
func main() {  
  // Bloque de la función main  
  x := 10  
  
  if x > 5 {  
    // Bloque del if  
    y := x * 2  
    fmt.Println(y)  
  }  
  
  // fmt.Println(y) // Error: y no es visible fuera del bloque if  
}
```

4. Declaración de Variables (Gramática)

```
<varDcl> ::= <var> <id_list> <type>  
          | <var> <id_list> <type> = <exp_list>
```

5. Ejemplos de Variables

```
var x int = 10
```

```
var y int
```

```
var w, z int = 1, 2
```

```
x = 20
```

```
// Ahora x vale 20
```

```
// y vale nil
```

```
// w vale 1 y z vale 2
```

```
// La longitud de la lista de ids y la de expresiones debe ser la misma.
```

6. Declaración corta

```
<shortValDcl> ::= <id_list> := <exp_list>
```

Ejemplo:

```
func main() {  
    x, y := 34, 68  
  
    fmt.Println(x)  
    fmt.Println(y)  
}
```

```
// Output esperado:
```

```
// 34
```

```
// 68
```

7. Constantes

`<consVarDcl> ::= <const> <id> <type> = <exp>`

Ejemplo:

```
const max int = 100
const pi float32 = 3.14
```

8. Ejemplo de Cortocircuito lógico

```
func esValido(x int32) bool {
    println("Evaluando función")
    return x > 0
}
```

```
var flag bool = false
```

```
// La función esValido no se ejecuta debido al cortocircuito
flag && esValido(10)
```

9. IF

`<ifStmt> ::= <if> <cond> { <stmts> }
| <if> <cond> { <stmts> } <else> { <stmts> }`

Ejemplo:

```
x := 10
```

```
if x > 0 {
    println("x es positivo")
} else {
    println("x es cero o negativo")
}
```

10. SWITCH

```
<switchStmt> ::= <switch> <exp> {  
    <case> <exp>:  
        <stmts>  
    ...  
    <default>:  
        <stmts>  
}
```

Ejemplo:

```
switch day {  
case 1:  
    fmt.Println("Lunes")  
case 2, 3:  
    fmt.Println("Martes o Miércoles")  
default:  
    fmt.Println("Otro día")  
}
```

11. FOR

```
<forStmt> ::= <for> <varDcl>; <cond>; <exp> { <stmts> }  
           | <for> <cond> { <stmts> }  
           | <for> { <stmts> }
```

Ejemplos:

```
for i := 0; i < 5; i++ {  
    fmt.Println(i)  
}
```

```
x := 3
```

```
for x > 0 {  
    fmt.Println(x)  
    x--  
}
```

```
for {  
    fmt.Println("Bucle infinito")  
    break
```

```
}
```

12. BREAK

```
for i := 0; i < 10; i++ {  
    if i == 5 {  
        break  
    }  
    fmt.Println(i)  
}
```

```
// Output esperado:  
// 0  
// 1  
// 2  
// 3  
// 4
```

13. CONTINUE

```
for i := 0; i < 5; i++ {  
    if i == 2 {  
        continue  
    }  
    fmt.Println(i)  
}
```

```
// Output esperado:  
// 0  
// 1  
// 3  
// 4
```

14. RETURN

```
func suma(a int, b int) int {  
    return a + b  
}
```

```
func main() {  
    resultado := suma(3, 4)  
    fmt.Println(resultado)  
}
```

```
// Output esperado:  
// 7
```

15. Arreglos simples

```
<arrayDecl> ::= <var> <id> [ <exp> ] ... <type>  
              | <var> <id> [ <exp> ] ... <type> = [ <exp> ] <type> { <exp>, ... }
```

Ejemplos:

```
var a [5]int
```

16. Arreglo inicializado

```
var b [3]int = [3]int{1, 2, 3}
```

17. Asignación en arreglos

```
var nums [4]int
```

```
nums[0] = 10  
nums[1] = 20
```

```
fmt.Println(nums[0])  
fmt.Println(nums[1])
```

```
// Output esperado:  
// 10  
// 20
```

18. Arreglos de distintos tipos

```
var letters [3]rune = [3]rune{'a', 'b', 'c'}
var names [2]string = [2]string{"Ana", "Luis"}
```

19. Arreglos multidimensionales

```
var m [2][3]int

var mat [2][2]int = [2][2]int{
    {1, 2},
    {3, 4},
}

var grid [2][3]int

grid[0][1] = 5
grid[1][2] = 8

fmt.Println(grid[0][1])
fmt.Println(grid[1][2])

// Output esperado:
// 5
// 8
```

20. For con arreglos

```
var m [2][2]int = [2][2]int{
    {1, 2},
    {3, 4},
}

for i := 0; i < 2; i++ {
    for j := 0; j < 2; j++ {
        fmt.Println(m[i][j])
    }
}

// Output esperado:
// 1
```



```
// 2
// 3
// 4
```

21. Función simple

```
func suma(a int, b int) int {
    return a + b
}
```

```
func main() {
    r := suma(3, 4)
    fmt.Println(r)
}
```

```
// Output esperado:
// 7
```

22. Funciones con múltiples retornos

```
func dividir(a int, b int) (int, bool) {
    if b == 0 {
        return 0, false
    }
    return a / b, true
}
```

```
func main() {
    resultado, ok := dividir(10, 2)

    if ok {
        fmt.Println(resultado)
    }
}
```

```
// Output esperado:
// 5
```

23. Punteros (paso por referencia)

// Ordenamiento SIN punteros (paso por valor)

```
func sort(a [5]int) [5]int {
    for i := 0; i < 5; i++ {
        for j := 0; j < 4; j++ {
            if a[j] > a[j+1] {
                temp := a[j]
                a[j] = a[j+1]
                a[j+1] = temp
            }
        }
    }
    return a
}
```

// Ordenamiento CON punteros (paso por referencia)

```
func sortRef(a *[5]int) {
    for i := 0; i < 5; i++ {
        for j := 0; j < 4; j++ {
            if a[j] > a[j+1] {
                temp := a[j]
                a[j] = a[j+1]
                a[j+1] = temp
            }
        }
    }
}
```

24. Uso de punteros

```
func main() {
    nums1 := [5]int{5, 3, 4, 1, 2}
    nums2 := [5]int{5, 3, 4, 1, 2}

    nums1 = sort(nums1)

    sortRef(&nums2)

    fmt.Println(nums1[0], nums1[1], nums1[2], nums1[3], nums1[4])
    fmt.Println(nums2[0], nums2[1], nums2[2], nums2[3], nums2[4])
}
```

// Output esperado:

```
// 1 2 3 4 5  
// 1 2 3 4 5
```

25. Builtin fmt.Println

```
fmt.Println("Resultado:", 10, true)
```

```
// Output esperado:  
// Resultado: 10 true
```

26. len()

```
var a [4]int  
var s string = "Hola"
```

```
fmt.Println(len(a))  
fmt.Println(len(s))
```

```
// Output esperado:  
// 4  
// 4
```

27. now()

```
var fecha string = now()  
fmt.Println(fecha)
```

```
// Output esperado (ejemplo):  
// 2026-01-21 10:30:00
```

28. substr()

```
var s string = "Compiladores"  
var sub string = substr(s, 0, 4)
```

```
fmt.Println(sub)
```

```
// Output esperado:
```

```
// Comp
```

29. typeof()

```
func main() {  
    x := 34  
    fmt.Println(typeof(r))
```

```
    // resultado
```

```
    // int32
```

```
}
```

30. Hoisting

```
func main() {  
    saludar()  
}
```

```
func saludar() {  
    fmt.Println("Hola")  
}
```

ENTRADAS Y SALIDAS OFICIALES:

entrada: archivo1_basico.go

salida:

=== INICIO DE CALIFICACION: FUNCIONALIDADES BASICAS ===

--- 1.1 DECLARACION LARGA ---

42 3.14 true 71 Golampi

--- 1.2 ASIGNACION DE VARIABLES ---

120 9.75 false 90 Actualizado

--- 1.3 FORMATO DE IDENTIFICADORES ---

Case sensitive: 1 2

--- 1.4 DECLARACION CORTA ---

7 2.5 true 88 Inferencia

--- 1.5 DECLARACION LARGA SIN INICIALIZAR ---

0 0 false 0

--- 1.6 DECLARACION MULTIPLE ---

10 20 Hola Mundo

--- 1.7 CONSTANTES ---

3.14159 1000

--- 1.8 MANEJO DE NIL ---

Impresion de nil: <nil>

Comparacion nil == nil: <nil>

--- 1.11 OPERACIONES ARITMETICAS ---

+: 40

-: 32

*: 56

/: 33

?: 2

--- 1.12 OPERACIONES RELACIONALES ---

==: false

!=: true

<: true

>: false

--- 1.13 OPERACIONES LOGICAS ---

true && false: false

true || false: true

!true: false

--- 1.14 CORTO CIRCUITO ---

AND: false

OR: true

--- 1.15 OPERADORES DE ASIGNACION ---

Resultado final: 22

=== FIN DE CALIFICACION: FUNCIONALIDADES BASICAS ===

Identificador	Tipo	Clase	Ambito	Valor	Fila	Columna
main	funcion	funcion	global	funcion	13	5
varInt	int32	variable	local	120	20	5
varFloat	float32	variable	local	9.75	21	5
varBool	bool	variable	local	false	22	5
varRune	rune	variable	local	90	23	5
varString	string	variable	local	Actualizado	24	5
identificador	int32	variable	local	1	42	5
Identificador	int32	variable	local	2	43	5
cInt	int	variable	local	7	50	1
cFloat	float64	variable	local	2.5	51	1
cBool	bool	variable	local	true	52	1
cRune	int32	variable	local	88	53	1
cString	string	variable	local	Inferencia	54	1
defInt	int32	variable	local	0	61	5
defFloat	float32	variable	local	0	62	5
defBool	bool	variable	local	false	63	5
defRune	rune	variable	local	0	64	5
defString	string	variable	local		65	5
m1	int32	variable	local	10	72	5
m2	int32	variable	local	20	72	9
m3	string	variable	local	Hola	73	1
m4	string	variable	local	Mundo	73	5
PI	float32	constante	local	3.14159	80	7
MAX	int32	constante	local	1000	81	7
r1	int32	variable	local	10	105	5
r2	int32	variable	local	20	106	5
t	bool	variable	local	true	116	5
f	bool	variable	local	false	117	5
divisor	int32	variable	local	0	126	5
ccAnd	bool	variable	local	false	127	1
ccOr	bool	variable	local	true	128	1
asig	int32	variable	local	22	136	5

entrada: archivo2_intermedio.go

salida:

=== INICIO DE CALIFICACION: ESTRUCTURAS DE CONTROL ===

--- 2.2 IF ---

El estudiante Ana tiene una nota mayor a 80

--- 2.3 IF ELSE ---

Clasificacion: EXCELENTE

--- 2.4 SWITCH CASE DEFAULT ---

Categoria 2: Intermedio

--- 2.5 FOR CLASICO ---

Iteracion: 1

Iteracion: 2

Iteracion: 3

Iteracion: 4

Iteracion: 5

--- 2.6 FOR CONDICIONAL ---

Cuenta regresiva: 10

Cuenta regresiva: 7

Cuenta regresiva: 4

Cuenta regresiva: 1

--- 2.7 FOR INFINITO ---

Intento: 1

Intento: 2

Intento: 3

--- 2.8 BREAK ---

1

2

3

4

5

6

Se encontro 7, se aplica break

--- 2.9 CONTINUE ---

Impar: 1

Impar: 3

Impar: 5

=== FIN DE CALIFICACION: ESTRUCTURAS DE CONTROL ===

Identificador	Tipo	Clase	Ambito	Valor	Fila	Columna
main	funcion	funcion	global	funcion	7	5
nota1	int	variable	local	85	10	1
nota2	int	variable	local	92	11	1
estudiante	string	variable	local	Ana	12	1
codigoCategoria	int	variable	local	2	40	1
i	int	variable	local	6	56	5
contador	int	variable	local	-2	64	1
intentos	int	variable	local	3	74	1
i	int	variable	local	7	87	5
j	int	variable	local	7	99	5

entrada: archivo3_funciones.go

salida:

=== INICIO DE CALIFICACION: FUNCIONES ===

--- 3.1 IMPRIMIR ARBOL ---

*

--- 3.2 CALCULAR VOLUMEN PIRAMIDE ---

Volumen: 500

--- 3.3 REFERENCIA: ORDENAMIENTO E INTERCAMBIO ---

Intercambio: 200 100

Ordenado: 11 12 22 25 64

--- 3.4 DIVISION MULTIPLE RETORNO ---

Cociente: 3 Residuo: 2

--- 3.5 POTENCIA RECURSIVA ---

2^8: 256

--- 3.6 EUCLIDES RECURSIVO ---

MCD: 6 Pasos: 4

--- 3.7 HOISTING ---

Funcion ejecutada por hoisting

--- 4.1 FMT.PRINTLN ---

Impresion directa de texto

--- 4.2 LEN ---

len(texto): 7

len(arrLen): 4

--- 4.3 NOW ---

Fecha actual: 2026-04-07 03:13:50

--- 4.4 SUBSTR ---

substr: Organizacion

--- 4.5 TYPEOF ---

int32: int32

float32: float32

bool: bool

string: string

=== FIN DE CALIFICACION: FUNCIONES ===

Identificador	Tipo	Clase	Ambito	Valor	Fila	Columna
main	funcion	funcion	global	funcion	7	5
imprimirArbol	funcion	funcion	global	funcion	102	5
calcularVolumenPiramide	funcion	funcion	global	funcion	108	5
intercambioValores	funcion	funcion	global	funcion	112	5
ordenamientoSeleccion	funcion	funcion	global	funcion	118	5
division	funcion	funcion	global	funcion	134	5
potencia	funcion	funcion	global	funcion	141	5
euclides	funcion	funcion	global	funcion	148	5
funcionDefinidaDespues	funcion	funcion	global	funcion	156	5
a	int32	variable	local	200	26	5
b	int32	variable	local	100	27	5
temp	int	variable	local	100	113	1
arr	[5]int32	variable	local	[11,12,22,25,64]	31	5
i	int	variable	local	4	119	5
min	int	variable	local	4	120	2
j	int	variable	local	5	121	6
temp	int	variable	local	64	127	3
min	int	variable	local	2	120	2
j	int	variable	local	5	121	6
temp	int	variable	local	25	127	3
min	int	variable	local	3	120	2
j	int	variable	local	5	121	6
temp	int	variable	local	25	127	3
min	int	variable	local	3	120	2
j	int	variable	local	5	121	6
cociente	int	variable	local	3	39	1
residuo	int	variable	local	2	39	11
valido	bool	variable	local	true	39	20
resultado	int	variable	local	6	152	1
pasos	int	variable	local	1	152	12
resultado	int	variable	local	6	152	1
pasos	int	variable	local	2	152	12
resultado	int	variable	local	6	152	1
pasos	int	variable	local	3	152	12
mcd	int	variable	local	6	54	1
pasos	int	variable	local	4	54	6
texto	string	variable	local	Golampi	73	1
arrLen	[4]int32	variable	local	[1,2,3,4]	75	1

entrada: archivo4_arreglos1d.go

salida:

=== INICIO DE CALIFICACION: ARREGLOS ===

--- 5.1 DECLARACION 1D INICIALIZADA Y NO INICIALIZADA ---

No inicializado pos0: 0

Inicializado pos2: 30

--- 5.2 ARREGLOS DE TIPOS ESTATICOS ---

1.1 false 67 Ana

--- 5.3 ACCESO Y MODIFICACION 1D ---

Original arregloInit[1]: 20

Modificado arregloInit[1]: 99

Longitud arregloInit: 5

--- 5.4 DECLARACION MULTIDIMENSIONAL ---

Matriz no inicializada [1][1]: 0

Matriz inicializada [0][0]: 1

--- 5.5 ACCESO Y MODIFICACION MULTIDIMENSIONAL ---

Original matrizNoInit[0][1]: 0

Modificado matrizNoInit[0][1]: 77

=== FIN DE CALIFICACION: ARREGLOS ===

Identificador	Tipo	Clase	Ambito	Valor	Fila	Columna
main	funcion	funcion	global	funcion	7	5
arregloNoInit	[5]int32	variable	local	[0,0,0,0,0]	14	5
arregloInit	[5]int32	variable	local	[10,99,30,40,50]	15	1
arrFloat	[3]float32	variable	local	[1.1,2.2,3.3]	23	1
arrBool	[3]bool	variable	local	[true,false,true]	24	1
arrRune	[3]rune	variable	local	[65,66,67]	25	1
arrString	[3]string	variable	local	["Ana","Luis","Maria"]	26	1
matrizNoInit	[2][2]int32	variable	local	[[0,77],[0,0]]	42	5
matrizInit	[2][2]int32	variable	local	[[1,2],[3,4]]	43	1

entrada: archivo5_arreglos_ndim.go

salida:

=== INICIO DE CALIFICACION: ARREGLOS N-D ===

--- 5.6 INDICE DE INESTABILIDAD ---

Indice: 25

--- 5.7 REGLA DE CRAMER ---

x, y: 1 1

--- 5.8 PROMEDIO DE CAPAS ---

Promedios capa 0: 2 6

Promedios capa 1: 3 7

--- 5.9 SOFTMAX ---

Fila 0: 0.0833333333 0.0833333333 0.8333333333

Fila 1: 0.8333333333 0.0833333333 0.0833333333

=== FIN DE CALIFICACION: ARREGLOS N-D ===

entrada: archivo6_avanzado.go

salida:

=== INICIO DE CALIFICACION: INTEGRACION ===

--- A.1 GESTION DE CALIFICACIONES ---

Ana - 85

Muy bueno

Luis - 92

Excelente

María - 78

Bueno

Carlos - 95

Excelente

Sofía - 88

Muy bueno

Mejor: Carlos 95

--- A.2 ANALISIS DE MATRICES ---

Matriz original:

12 15 18

22 25 28

32 35 38

Después de multiplicar por 2:

24 30 36

44 50 56

64 70 76

--- A.3 BUSQUEDA MATRICIAL ---

303 encontrado en [2][2]

--- A.4 SERIES NUMERICAS ---

Primo: 2

Primo: 3

Primo: 5

Primo: 7

Primo: 11

Fibonacci: 0 1 1 2 3 5

--- A.5 ANALISIS DE TEXTO ---

Texto 0 : Organizacion de Lenguajes

Longitud: 25

Vocales: 11

Tipo: string

Primeros 5: Organ

Texto 1 : Compiladores

Longitud: 12

Vocales: 5

Tipo: string

Primeros 5: Comp

Texto 2 : Analisis Semantico

Longitud: 18

Vocales: 8

Tipo: string

Primeros 5: Anali

--- A.6 MATEMATICAS RECURSIVAS ---

Suma 1..100: 5050

Suma 10..50: 1230

Datos: 45 12 67 23 89 34 56

3er menor: 34

--- A.7 VALIDACION Y CONTROL ---

Rango [10,60]: OK

Rango [20,40]: FUERA

--- A.8 AMBITOS ANIDADOS ---

Nivel 1: 100 Nivel 2: 200

Nivel 1: 100 Nivel 2: 200 Nivel 3: 300

Nivel 4 (i= 0): 400

Nivel 4 (i= 1): 401

--- A.9 FUNCIONES EMBEBIDAS ---

Texto: Golampi 2026

Longitud: 12

Subcadena: Golampi

Hora: 2026-04-07 03:17:49

Tipo: string

--- A.10 CASOS LIMITE ---

Array unitario: 42

String vacío longitud: 0

Rango int32: -2147483648 a 2147483647

Comparación: OK

=== FIN DE CALIFICACION: INTEGRACION ===

Identificador	Tipo	Clase	Ambito	Valor	Fila	Columna
procesarMatriz	funcion	funcion	global	funcion	20	5
buscarEnMatriz	funcion	funcion	global	funcion	28	5
esPrimo	funcion	funcion	global	funcion	39	5
generarFibonacci	funcion	funcion	global	funcion	57	5
contarVocales	funcion	funcion	global	funcion	65	5
sumaRango	funcion	funcion	global	funcion	79	5
encontrarKesimoMenor	funcion	funcion	global	funcion	89	5
validarRango	funcion	funcion	global	funcion	111	5
main	funcion	funcion	global	funcion	124	5
estudiantes	[5]string	variable	local	["Ana","Luis","Mar'u00eda","Carlos","Sofia"]	131	5
calificaciones	[5]int32	variable	local	[85,92,78,95,88]	132	5
i	int	variable	local	5	134	5
mejorCalif	int32	variable	local	95	147	5
indiceMejor	int32	variable	local	3	148	5
i	int	variable	local	5	149	5
datos	[3][3]int32	variable	local	[[24,30,36],[44,50,56],[64,70,76]]	161	5
i	int	variable	local	3	168	5
i	int	variable	local	3	21	5
j	int	variable	local	3	22	6
j	int	variable	local	3	22	6
j	int	variable	local	3	22	6
i	int	variable	local	3	174	5
tabla	[4][4]int32	variable	local	[[101,102,103,104],[201,202,203,204],[301,302,303,304],[401,402,403,404]]	182	5
i	int	variable	local	2	29	5
j	int	variable	local	4	30	6
j	int	variable	local	4	30	6
j	int	variable	local	2	30	6
fila	int	variable	local	2	189	5
col	int	variable	local	2	189	11
encon	bool	variable	local	true	189	16
primosCont	int32	variable	local	5	198	5
primosEnc	int32	variable	local	5	199	5

n	int	variable	local	12	200	5
i	int	variable	local	5	49	5
i	int	variable	local	5	49	5
i	int	variable	local	5	49	5
fib	[10]int32	variable	local	[0,1,1,2,3,5,8,13,21,34]	208	5
i	int	variable	local	10	60	5
textos	[3]string	variable	local	["Organizacion de Lenguajes","Compiladores","Analisis Semantico"]	216	5
i	int	variable	local	3	222	5
contador	int32	variable	local	11	66	5
i	int	variable	local	25	67	5
letra	rune	variable	local	79	68	6
letra	rune	variable	local	114	68	6
letra	rune	variable	local	103	68	6
letra	rune	variable	local	97	68	6
letra	rune	variable	local	110	68	6
letra	rune	variable	local	105	68	6
letra	rune	variable	local	122	68	6
letra	rune	variable	local	97	68	6
letra	rune	variable	local	99	68	6
letra	rune	variable	local	105	68	6
letra	rune	variable	local	111	68	6
letra	rune	variable	local	110	68	6
letra	rune	variable	local	32	68	6
letra	rune	variable	local	100	68	6
letra	rune	variable	local	101	68	6
letra	rune	variable	local	32	68	6
letra	rune	variable	local	76	68	6
letra	rune	variable	local	101	68	6
letra	rune	variable	local	110	68	6
letra	rune	variable	local	103	68	6
letra	rune	variable	local	117	68	6
letra	rune	variable	local	97	68	6
letra	rune	variable	local	106	68	6

letra	rune	variable	local	101	68	6
letra	rune	variable	local	115	68	6
sub	string	variable	local	Organ	229	7
contador	int32	variable	local	5	66	5
i	int	variable	local	12	67	5
letra	rune	variable	local	67	68	6
letra	rune	variable	local	111	68	6
letra	rune	variable	local	109	68	6
letra	rune	variable	local	112	68	6
letra	rune	variable	local	105	68	6
letra	rune	variable	local	108	68	6
letra	rune	variable	local	97	68	6
letra	rune	variable	local	100	68	6
letra	rune	variable	local	111	68	6
letra	rune	variable	local	114	68	6
letra	rune	variable	local	101	68	6
letra	rune	variable	local	115	68	6
sub	string	variable	local	Compi	229	7
contador	int32	variable	local	8	66	5
i	int	variable	local	18	67	5
letra	rune	variable	local	65	68	6
letra	rune	variable	local	110	68	6
letra	rune	variable	local	97	68	6
letra	rune	variable	local	108	68	6
letra	rune	variable	local	105	68	6
letra	rune	variable	local	115	68	6
letra	rune	variable	local	105	68	6
letra	rune	variable	local	115	68	6
letra	rune	variable	local	32	68	6
letra	rune	variable	local	83	68	6
letra	rune	variable	local	101	68	6
letra	rune	variable	local	109	68	6
letra	rune	variable	local	97	68	6

letra	rune	variable	local	110	68	6
letra	rune	variable	local	116	68	6
letra	rune	variable	local	105	68	6
letra	rune	variable	local	99	68	6
letra	rune	variable	local	111	68	6
sub	string	variable	local	Anali	229	7
suma1	int32	variable	local	5050	238	5
suma2	int32	variable	local	1230	241	5
datos2	[7]int32	variable	local	[45,12,67,23,89,34,56]	244	5
temp	[7]int32	variable	local	[12,23,34,45,56,67,89]	90	5
i	int	variable	local	7	91	5
i	int	variable	local	7	95	5
j	int	variable	local	6	96	6
aux	int32	variable	local	45	98	8
aux	int32	variable	local	67	98	8
aux	int32	variable	local	89	98	8
aux	int32	variable	local	89	98	8
j	int	variable	local	6	96	6
aux	int32	variable	local	45	98	8
aux	int32	variable	local	67	98	8
aux	int32	variable	local	67	98	8
j	int	variable	local	6	96	6
aux	int32	variable	local	45	98	8
j	int	variable	local	6	96	6
j	int	variable	local	6	96	6
j	int	variable	local	6	96	6
j	int	variable	local	6	96	6
entrada	[5]int32	variable	local	[15,22,35,48,56]	252	5
i	int	variable	local	5	112	5
i	int	variable	local	0	112	5
nivel1	int32	variable	local	100	265	5
nivel2	int32	variable	local	200	267	6
nivel3	int32	variable	local	300	270	7
i	int	variable	local	2	272	7
nivel4	int32	variable	local	400	273	8
nivel4	int32	variable	local	401	273	8
texto	string	variable	local	Golampi 2026	283	5
unitario	[1]int32	variable	local	[42]	294	5
vacio	string	variable	local		297	5
minInt	int32	variable	local	-2147483648	300	5
unitario	[1]int32	variable	local	[42]	294	5
vacio	string	variable	local		297	5
minInt	int32	variable	local	-2147483648	300	5
maxInt	int32	variable	local	2147483647	301	5

Proyecto 2

UNIVERSIDAD SAN CARLOS DE GUATEMALA

Organización de Lenguajes y Compiladores 2

PROYECTO 2

Golampi Compiler

Ponderacion	50 puntos
Tiempo Estimado	40 horas
Inicio	13/03/2026
Entrega	29/04/2026
Auxiliar	rubenralda (Seccion B)

1. Resumen Ejecutivo

El Proyecto 2 consiste en el desarrollo de un compilador completo para el lenguaje Golampi, un lenguaje academico con sintaxis inspirada en Golang. A diferencia del Proyecto 1 (interprete), este proyecto eleva el nivel de complejidad al requerir la generacion de codigo ensamblador ARM64 real, que se ensambla y ejecuta mediante QEMU.

El sistema implementa las fases formales de un compilador:

- Analisis Lexico: tokenizacion via ANTLR4
- Analisis Sintactico: construccion del arbol via ANTLR4 Parser
- Analisis Semantico: tabla de simbolos, validacion de tipos y ambitos via Visitor PHP
- Generacion deCodigo: traduccion directa a ensamblador ARM64 (AArch64)
- Ensamblaje y Ejecucion: mediante aarch64-linux-gnu-as + qemu-aarch64

La arquitectura es monolitica cliente/servidor. El backend en PHP orquesta todo el proceso de compilacion y la interfaz web (HTML/CSS/JS) actua como editor y visor de resultados.

2. Diferencias Clave vs Proyecto 1 (Interprete)

Aspecto	Proyecto 1 - Interprete	Proyecto 2 - Compilador
Resultado final	Ejecucion directa en PHP	Codigo ensamblador ARM64 (.s)
Consola muestra	Salida del programa	Codigo ARM64 generado
Ejecucion	PHP interpreta el AST	QEMU ejecuta el binario ARM64
Fase extra	Ninguna	Generacion de codigo + ensamblaje
Panel reportes	Errores + Tabla simbolos	Errores + Tabla simbolos + Descargar ARM64
Stack memoria	Variables en PHP arrays	Stack frame AArch64 (sp, x29, x30)
Ponderacion	45 puntos	50 puntos

3. Arquitectura del Sistema

3.1 Flujo de Compilacion

El compilador sigue este flujo end-to-end:

Pas o	Capa	Accion
1	Frontend (JS)	Usuario escribe codigo Golampi y presiona Compilar
2	Backend (PHP)	execute.php recibe el codigo via POST
3	ANTLR4	Lexer tokeniza; Parser construye el CST
4	Visitor PHP	Recorre el CST: analisis semantico + generacion ARM64
5	ARM64 Generator	Emite instrucciones AArch64 al archivo output.s
6	Ensamblador	aarch64-linux-gnu-as ensambla el .s a binario
7	QEMU	qemu-aarch64 ejecuta el binario y captura stdout
8	Backend (PHP)	Serializa: { arm64_code, output, errors, symbols }
9	Frontend (JS)	Muestra ARM64 en consola; habilita descarga de reportes

3.2 Modelo de Memoria ARM64

El compilador implementa el modelo de memoria AArch64:

- Stack: variables locales, parametros, direcciones de retorno, registros preservados
- Heap: cadenas y arreglos (mediante malloc en ensamblador o section .data)
- Convencion de llamadas: x0-x7 para parametros, x0 para retorno, x29=frame pointer, x30=link register

3.3 Separacion de Responsabilidades

Capa	Responsabilidad
Presentacion (Frontend)	Editor de codigo, consola ARM64, botones, descarga reportes
Logica compilador (Backend PHP)	Orquesta todas las fases del compilador
ANTLR4 (Lexico/Sintactico)	Lexer + Parser + generacion del CST
Visitor PHP (Semantico)	Tabla de simbolos, validacion tipos, offsets para ARM64
CodeGen PHP (ARM64)	Traduce construcciones Golampi a instrucciones AArch64
Ensamblaje/Ejecucion	aarch64-linux-gnu-as + qemu-aarch64

4. Estructura de Carpetas del Proyecto

Basada en el Proyecto 1 (interprete) como punto de partida, adaptada para incluir la generacion de codigo ARM64:

4.1 Arbol de Directorios

```
golampi-compiler/
├── grammar/                                # Archivos ANTLR4
│   └── Golampi.g4                          # Gramatica del lenguaje (REUTILIZAR de
P1)
├── backend/                                # Toda la logica del compilador en PHP
│   ├── api/
│   │   └── execute.php                    # Endpoint principal: recibe codigo ->
retorna JSON
│   └── generated/                          #Codigo PHP generado por ANTLR4 (no
editar)
│       ├── GolampiLexer.php
│       ├── GolampiParser.php
│       ├── GolampiBaseVisitor.php
│       └── GolampiVisitor.php
│
│   ├── interpreter/                       # Nucleo del compilador
│   │   ├── Visitor.php                   # Visitor: semantica + delega a CodeGen
│   │   ├── CodeGen.php                   # NUEVO: generador de codigo ARM64
│   │   ├── Environment.php               # Entornos anidados con parent pointer
│   │   ├── FuncionUsuario.php            # Representacion de funciones
│   │   ├── FlowTypes.php                 # Excepciones para break/continue/return
│   │   ├── Invocable.php                 # Interfaz para funciones
│   │   └── ErrorHandler.php              # Recolector de errores
│   └── arm64/                             # NUEVO: utilidades ARM64
│       ├── ArmEmitter.php                # Clase para emitir instrucciones ARM64
│       ├── LabelManager.php              # Generador de etiquetas unicas
│       ├── RegisterAllocator.php         # Asignacion de registros
│       └── Assembler.php                 # Invoca as + ld + qemu
│
│   ├── models/
│   │   ├── Symbol.php                   # Simbolo (id, tipo, valor, ambito,
linea, offset)
│   │   └── ErrorEntry.php                # Entrada de error
│   └── reports/
│       ├── SymbolTableReport.php
│       └── ErrorReport.php
└── frontend/                              # Interfaz de usuario
    ├── index.html                        # Pagina principal
    ├── css/
    │   └── style.css
    └── js/
```


editor.js	# Logica del editor de codigo
api.js	# Comunicacion con backend
reports.js	# Descarga de reportes
docs/	# Documentacion tecnica
README.md	
gramatica.md	
diagrama-clases.md	
manual-usuario.md	
tests/	# Casos de prueba
archivo1_basico.gol	
archivo2_intermedio.gol	
archivo3_funciones.gol	
archivo4_arreglos1d.gol	
archivo5_arreglos_ndim.gol	
archivo6_avanzado.gol	
output/	# NUEVO: archivos generados en runtime
program.s	# Ensamblador ARM64 generado
program.o	# Objeto ensamblado
program	# Ejecutable ARM64
vendor/	# Dependencias PHP (ANTLR4 runtime)
composer.json	
.htaccess	
build.sh	# Script para compilar la gramatica

4.2 Archivos Nuevos vs Reutilizados del Proyecto 1

Archivo	Estado	Descripcion
grammar/Golampi.g4	REUTILIZAR	Gramatica del P1, posiblemente extender
backend/generated/*	REGENERAR	Re-generar desde la gramatica actualizada
backend/interpreter/Environment.php	REUTILIZAR	Mismos entornos anidados
backend/interpreter/FuncionUsuario.php	REUTILIZAR	Misma estructura de funciones
backend/interpreter/FlowTypes.php	REUTILIZAR	Break/continue/return
backend/interpreter/ErrorHandler.php	REUTILIZAR	Recolector de errores
backend/interpreter/Visitor.php	ADAPTAR	Agregar offsets y delegacion a CodeGen
backend/interpreter/CodeGen.php	NUEVO	Nucleo: genera instrucciones ARM64
backend/arm64/ArmEmitter.php	NUEVO	Clase de emision de instrucciones
backend/arm64/LabelManager.php	NUEVO	Etiquetas unicas para el ensamblador
backend/arm64/Assembler.php	NUEVO	Invoca as, ld, qemu en el servidor
backend/models/Symbol.php	ADAPTAR	Agregar campo 'offset' para stack frame

Archivo	Estado	Descripcion
backend/api/execute.php	ADAPTAR	Retornar arm64_code ademas de output/errors
frontend/index.html	ADAPTAR	Consola muestra ARM64; boton Descargar ARM64
frontend/js/api.js	ADAPTAR	Mostrar codigo ARM64 en consola
frontend/js/reports.js	ADAPTAR	Agregar descarga de archivo .s
output/	NUEVO	Directorio para archivos generados en runtime

5. Referencia Rapida del Lenguaje Golampi

5.1 Tipos de Datos

Tipo	Descripcion	Valor por Defecto
int32	Entero con signo de 32 bits. Rango: -2^{31} a $2^{31} - 1$	0
float32	Punto flotante IEEE-754 de 32 bits	0.0
bool	Valor logico: true o false	false
rune	Caracter Unicode (alias de int32), entre comillas simples	'\u0000'
string	Cadena de texto Unicode, entre comillas dobles	""

5.2 Declaracion de Variables

```
// Declaracion larga
var x int32 = 10
var y int32           // valor por defecto: 0
var w, z int32 = 1, 2 // multiples variables

// Declaracion corta (solo dentro de funciones)
x, y := 34, 68

// Constantes
const max int32 = 100
const pi float32 = 3.14
```

5.3 Operadores Aritméticos y Reglas de Tipos

Suma (+):

+	int32	float32	bool	rune	string
int32	int32	float32		int32	
float32	float32	float32		float32	
bool					
rune	int32	float32		int32	
string					string

Multiplicacion (*): $\text{int32} * \text{string} = \text{string}$ (repeticion). Modulo (%) solo entre int32 y rune.

5.4 Estructuras de Control

```
// IF
```

```

if x > 0 {
    fmt.Println("positivo")
} else {
    fmt.Println("cero o negativo")
}

// SWITCH
switch day {
case 1:
    fmt.Println("Lunes")
case 2, 3:
    fmt.Println("Martes o Miercoles")
default:
    fmt.Println("Otro dia")
}

// FOR clasico
for i := 0; i < 5; i++ {
    fmt.Println(i)
}

// FOR condicional (tipo while)
x := 3
for x > 0 {
    fmt.Println(x)
    x--
}

// FOR infinito
for {
    fmt.Println("Bucle infinito")
    break
}

```

5.5 Funciones

```

// Funcion simple
func suma(a int32, b int32) int32 {
    return a + b
}

// Multiples retornos
func dividir(a int32, b int32) (int32, bool) {
    if b == 0 {
        return 0, false
    }
    return a / b, true
}

// Paso por referencia (punteros)
func sortRef(a *[5]int32) {
    for i := 0; i < 5; i++ {

```

```

        for j := 0; j < 4; j++ {
            if a[j] > a[j+1] {
                temp := a[j]
                a[j] = a[j+1]
                a[j+1] = temp
            }
        }
    }
}

func main() {
    r := suma(3, 4)
    fmt.Println(r)    // 7

    resultado, ok := dividir(10, 2)
    if ok {
        fmt.Println(resultado)    // 5
    }

    nums := [5]int32{5, 3, 4, 1, 2}
    sortRef(&nums)
    fmt.Println(nums[0], nums[1], nums[2], nums[3], nums[4])    // 1 2 3 4 5
}

```

5.6 Arreglos

```

// Declaracion simple (valor por defecto)
var a [5]int32

// Declaracion inicializada
var b [3]int32 = [3]int32{1, 2, 3}

// Arreglo de distintos tipos
var letters [3]rune = [3]rune{'a', 'b', 'c'}
var names [2]string = [2]string{"Ana", "Luis"}

// Asignacion y acceso
var nums [4]int32
nums[0] = 10
nums[1] = 20
fmt.Println(nums[0])    // 10
fmt.Println(nums[1])    // 20

// Multidimensional
var mat [2][2]int32 = [2][2]int32{
    {1, 2},
    {3, 4},
}

var grid [2][3]int32
grid[0][1] = 5
grid[1][2] = 8

```

```

fmt.Println(grid[0][1])    // 5
fmt.Println(grid[1][2])    // 8

// For con arreglos
for i := 0; i < 2; i++ {
    for j := 0; j < 2; j++ {
        fmt.Println(mat[i][j])
    }
}
// Output: 1 2 3 4

```

5.7 Funciones Embebidas

Funcion	Parametros	Retorno	Descripcion
fmt.Println(...)	Variadic	void	Imprime valores separados por espacio + \n
len(x)	string array	int32	Longitud de cadena o arreglo
now()	ninguno	string	Fecha y hora actual: YYYY-MM-DD HH:MM:SS
substr(s, i, n)	string, int32, int32	string	Subcadena desde indice i con longitud n
typeof(x)	cualquier tipo	string	Nombre del tipo como string: int32, float32, etc.

```

fmt.Println("Resultado:", 10, true)    // Resultado: 10 true

var a [4]int32
var s string = "Hola"
fmt.Println(len(a))    // 4
fmt.Println(len(s))    // 4

var fecha string = now()
fmt.Println(fecha)    // 2026-04-09 10:30:00

var sub string = substr("Compiladores", 0, 4)
fmt.Println(sub)    // Comp

x := 34
fmt.Println(typeof(x)) // int32

```

5.8 Cortocircuito Logico

```

func esValido(x int32) bool {
    fmt.Println("Evaluando funcion")
    return x > 0
}

var flag bool = false

```

```
// esValido NO se ejecuta por cortocircuito AND
flag && esValido(10)

// En OR: si a es true, b no se evalua
// En AND: si a es false, b no se evalua
```

5.9 Hoisting de Funciones

```
func main() {
    saludar()    // Funciona aunque saludar esta definida abajo
}

func saludar() {
    fmt.Println("Hola")
}
```

6. Generacion deCodigo ARM64

6.1 Convenciones AArch64

Registro	Uso
x0 - x7	Primeros 8 parametros de funcion
x0	Valor de retorno (si cabe en 64 bits)
x0 - x1	Retorno de hasta 128 bits
x19 - x28	Registros preservados por la funcion (callee-saved)
x29 (fp)	Frame Pointer
x30 (lr)	Link Register (direccion de retorno)
sp	Stack Pointer
xzr / wzr	Registro cero: lectura siempre 0, escritura descartada
w0 - w30	Version 32 bits de los registros x0-x30

6.2 Instrucciones Esenciales

```
# Aritmetica
add x3, x1, x2      // x3 = x1 + x2
sub x3, x1, x2      // x3 = x1 - x2
mul x3, x1, x2      // x3 = x1 * x2
sdiv x3, x1, x2     // x3 = x1 / x2 (signed)
add x3, x3, #1      // x3++ (no hay inc en ARM64)
sub x3, x3, #1      // x3--

# Memoria
ldr x0, [x1]        // cargar 64 bits desde [x1]
str x0, [x1]        // guardar 64 bits en [x1]
ldr x2, [x1, #16]   // cargar con desplazamiento
ldrb w3, [x1]       // cargar 8 bits

# Stack
sub sp, sp, #16     // reservar espacio
add sp, sp, #16     // liberar espacio
stp x29, x30, [sp, #-16]! // guardar fp y lr
ldp x29, x30, [sp], #16 // restaurar fp y lr

# Control de flujo
cmp x0, x1          // comparar (actualiza flags)
b.eq etiqueta       // salto si igual
b.ne etiqueta       // salto si no igual
b.lt etiqueta       // salto si menor
b.le etiqueta       // salto si menor o igual
b.gt etiqueta       // salto si mayor
b etiqueta          // salto incondicional
cbz x0, etiqueta    // salto si x0 == 0
```



```

cbnz x0, etiqueta    // salto si x0 != 0
bl funcion            // llamada a funcion (guarda x30)
ret                  // retorno (usa x30)

```

6.3 Ejemplo Completo: if/else en ARM64

Golampi:

```

func main() {
    a := 8
    b := 7
    var result int32
    if a > b {
        result = a
    } else {
        result = b
    }
    fmt.Println(result)
}

```

ARM64 generado:

```

.section .data
buffer: .ascii "0\n"

.section .text
.align 2
.global _start
_start:
    # a = 8
    mov x0, #8
    # b = 7
    mov x1, #7
    # if (a > b)
    cmp x0, x1
    b.le else_branch
    # result = a
    mov x2, x0
    b end_if
else_branch:
    # result = b
    mov x2, x1
end_if:
    # Convertir a ASCII y escribir
    add x3, x2, #48
    adrp x4, buffer
    add x4, x4, :lo12:buffer
    strb w3, [x4]
    mov x0, #1           # stdout
    mov x1, x4           # direccion buffer
    mov x2, #2           # longitud
    mov x8, #64          # syscall write
    svc #0
    # exit(0)

```

```
mov x0, #0
mov x8, #93
svc #0
```

6.4 Comentarios en Ensamblador ARM64

El código ARM64 generado debe estar bien comentado:

```
# Comentario de una línea (GNU gas)
/* Comentario
   multilinea */
```

Secciones a documentar obligatoriamente:

- Inicio y fin de cada función
- Manejo del stack frame (prologo y epilogo)
- Uso de registros específicos
- Secciones críticas del flujo de control

6.5 Flujo de Ensamblaje y Ejecución

```
# 1. El compilador genera el .s
# php execute.php -> genera output/program.s

# 2. Ensamblar
aarch64-linux-gnu-as -o output/program.o output/program.s

# 3. Enlazar
aarch64-linux-gnu-ld -o output/program output/program.o

# 4. Ejecutar con QEMU
qemu-aarch64 output/program
```

7. Reportes Requeridos

7.1 Reporte de Errores

Debe detectar y reportar errores sin detener el analisis al primer fallo:

- Lexico: simbolos no reconocidos
- Sintactico: construcciones incompletas o mal formadas
- Semantico: variables no declaradas, redeclaraciones, incompatibilidad de tipos

#	Tipo	Descripcion	Linea	Columna
1	Lexico	Simbolo no reconocido: @	12	8
2	Sintactico	Se esperaba ';' despues de la instruccion	25	14
3	Semantico	Variable 'x' no declarada en el ambito actual	40	5
4	Semantico	Identificador 'y' ya ha sido declarado	45	2
5	Semantico	Operacion invalida entre 'int32' y 'string'	50	13

7.2 Tabla de Simbolos

Contiene todos los identificadores declarados con su tipo, valor, ambito y ubicacion:

Identificador	Tipo	Ambito	Valor	Offset	Linea	Columna
bubbleSort	funcion	global	-	-	1	1
arr	arreglo	bubbleSort	-	0	1	18
n	int32	bubbleSort	-	80	1	23
i	int32	bubbleSort	0	88	5	9
j	int32	bubbleSort	0	96	6	9
temp	int32	bubbleSort	-	104	7	9

7.3Codigo ARM64 Descargable

El panel de reportes debe incluir boton para descargar el archivo .s con el codigo ensamblador ARM64 generado. Este archivo debe estar disponible siempre que haya una compilacion exitosa.

8. Interfaz Grafica (GUI)

8.1 Componentes Obligatorios

Componente	Descripcion
Barra de acciones	Botones: Nuevo/Limpiar, Cargar archivo, Guardar codigo, Compilar, Limpiar consola
Editor de codigo	Textarea multilínea para escribir codigo Golampi
Consola de salida	Muestra el codigo ARM64 generado (exito) o errores (fallo)
Panel de reportes	Botones: Ver Errores, Ver Tabla de Simbolos, Descargar ARM64

8.2 Diferencia Importante con el Proyecto 1

Proyecto 1 (Interprete)	Proyecto 2 (Compilador)
Consola muestra: output del programa (fmt.Println)	Consola muestra: codigo ARM64 generado
Panel: Errores + Tabla simbolos	Panel: Errores + Tabla simbolos + Descargar .s
La app ejecuta el programa	La app muestra el ARM64; QEMU lo ejecuta en backend

9. Entradas y Salidas Oficiales de Calificacion (Proyecto 1)

Los siguientes son los archivos de prueba y salidas esperadas que se usaran en la calificacion. El compilador debe generar codigo ARM64 que al ejecutarse produzca exactamente estas salidas.

archivo1_basico.go - Funcionalidades Basicas

```
=== INICIO DE CALIFICACION: FUNCIONALIDADES BASICAS ===

--- 1.1 DECLARACION LARGA ---
42 3.14 true 71 Golampi

--- 1.2 ASIGNACION DE VARIABLES ---
120 9.75 false 90 Actualizado

--- 1.3 FORMATO DE IDENTIFICADORES ---
Case sensitive: 1 2

--- 1.4 DECLARACION CORTA ---
7 2.5 true 88 Inferencia

--- 1.5 DECLARACION LARGA SIN INICIALIZAR ---
0 0 false 0

--- 1.6 DECLARACION MULTIPLE ---
10 20 Hola Mundo

--- 1.7 CONSTANTES ---
3.14159 1000

--- 1.8 MANEJO DE NIL ---
Impresion de nil: <nil>
Comparacion nil == nil: <nil>

--- 1.11 OPERACIONES ARITMETICAS ---
+: 40
-: 32
*: 56
/: 33
%: 2

--- 1.12 OPERACIONES RELACIONALES ---
==: false
!=: true
<: true
>: false

--- 1.13 OPERACIONES LOGICAS ---
true && false: false
true || false: true
```

```
!true: false

--- 1.14 CORTO CIRCUITO ---
AND: false
OR: true

--- 1.15 OPERADORES DE ASIGNACION ---
Resultado final: 22

=== FIN DE CALIFICACION: FUNCIONALIDADES BASICAS ===
```

archivo2_intermedio.go - Estructuras de Control

```
=== INICIO DE CALIFICACION: ESTRUCTURAS DE CONTROL ===

--- 2.2 IF ---
El estudiante Ana tiene una nota mayor a 80

--- 2.3 IF ELSE ---
Clasificacion: EXCELENTE

--- 2.4 SWITCH CASE DEFAULT ---
Categoria 2: Intermedio

--- 2.5 FOR CLASICO ---
Iteracion: 1
Iteracion: 2
Iteracion: 3
Iteracion: 4
Iteracion: 5

--- 2.6 FOR CONDICIONAL ---
Cuenta regresiva: 10
Cuenta regresiva: 7
Cuenta regresiva: 4
Cuenta regresiva: 1

--- 2.7 FOR INFINITO ---
Intento: 1
Intento: 2
Intento: 3

--- 2.8 BREAK ---
1
2
3
4
5
6
Se encontro 7, se aplica break
```

```
--- 2.9 CONTINUE ---
Impar: 1
Impar: 3
Impar: 5

=== FIN DE CALIFICACION: ESTRUCTURAS DE CONTROL ===
```

archivo3_funciones.go - Funciones y Builtins

```
=== INICIO DE CALIFICACION: FUNCIONES ===

--- 3.1 IMPRIMIR ARBOL ---
  *
  ***
  *****

--- 3.2 CALCULAR VOLUMEN PIRAMIDE ---
Volumen: 500

--- 3.3 REFERENCIA: ORDENAMIENTO E INTERCAMBIO ---
Intercambio: 200 100
Ordenado: 11 12 22 25 64

--- 3.4 DIVISION MULTIPLE RETORNO ---
Cociente: 3 Residuo: 2

--- 3.5 POTENCIA RECURSIVA ---
2^8: 256

--- 3.6 EUCLIDES RECURSIVO ---
MCD: 6 Pasos: 4

--- 3.7 HOISTING ---
Funcion ejecutada por hoisting

--- 4.1 FMT.PRINTLN ---
Impresion directa de texto

--- 4.2 LEN ---
len(texto): 7
len(arrLen): 4

--- 4.3 NOW ---
Fecha actual: 2026-04-07 03:13:50

--- 4.4 SUBSTR ---
substr: Organizacion

--- 4.5 TYPEOF ---
int32: int32
float32: float32
```

```
bool: bool
string: string

=== FIN DE CALIFICACION: FUNCIONES ===
```

archivo4_arreglos1d.go - Arreglos 1D

```
=== INICIO DE CALIFICACION: ARREGLOS ===

--- 5.1 DECLARACION 1D INICIALIZADA Y NO INICIALIZADA ---
No inicializado pos0: 0
Inicializado pos2: 30

--- 5.2 ARREGLOS DE TIPOS ESTATICOS ---
1.1 false 67 Ana

--- 5.3 ACCESO Y MODIFICACION 1D ---
Original arregloInit[1]: 20
Modificado arregloInit[1]: 99
Longitud arregloInit: 5

--- 5.4 DECLARACION MULTIDIMENSIONAL ---
Matriz no inicializada [1][1]: 0
Matriz inicializada [0][0]: 1

--- 5.5 ACCESO Y MODIFICACION MULTIDIMENSIONAL ---
Original matrizNoInit[0][1]: 0
Modificado matrizNoInit[0][1]: 77

=== FIN DE CALIFICACION: ARREGLOS ===
```

archivo5_arreglos_ndim.go - Arreglos N-Dimensionales

```
=== INICIO DE CALIFICACION: ARREGLOS N-D ===

--- 5.6 INDICE DE INESTABILIDAD ---
Indice: 25

--- 5.7 REGLA DE CRAMER ---
x, y: 1 1

--- 5.8 PROMEDIO DE CAPAS ---
Promedios capa 0: 2 6
Promedios capa 1: 3 7

--- 5.9 SOFTMAX ---
Fila 0: 0.0833333333 0.0833333333 0.8333333333
Fila 1: 0.8333333333 0.0833333333 0.0833333333

=== FIN DE CALIFICACION: ARREGLOS N-D ===
```


archivo6_avanzado.go - Integracion

```
=== INICIO DE CALIFICACION: INTEGRACION ===
```

```
--- A.1 GESTION DE CALIFICACIONES ---
```

```
Ana - 85
    Muy bueno
Luis - 92
    Excelente
Maria - 78
    Bueno
Carlos - 95
    Excelente
Sofia - 88
    Muy bueno
Mejor: Carlos 95
```

```
--- A.2 ANALISIS DE MATRICES ---
```

```
Matriz original:
12 15 18
22 25 28
32 35 38
Despues de multiplicar por 2:
24 30 36
44 50 56
64 70 76
```

```
--- A.3 BUSQUEDA MATRICIAL ---
```

```
303 encontrado en [ 2 ][ 2 ]
```

```
--- A.4 SERIES NUMERICAS ---
```

```
Primo: 2
Primo: 3
Primo: 5
Primo: 7
Primo: 11
Fibonacci: 0 1 1 2 3 5
```

```
--- A.5 ANALISIS DE TEXTO ---
```

```
Texto 0 : Organizacion de Lenguajes
    Longitud: 25
    Vocales: 11
    Tipo: string
    Primeros 5: Organ
Texto 1 : Compiladores
    Longitud: 12
    Vocales: 5
    Tipo: string
    Primeros 5: Comp
Texto 2 : Analisis Semantico
    Longitud: 18
    Vocales: 8
```

```
Tipo: string
Primeros 5: Anali

--- A.6 MATEMATICAS RECURSIVAS ---
Suma 1..100: 5050
Suma 10..50: 1230
Datos: 45 12 67 23 89 34 56
3er menor: 34

--- A.7 VALIDACION Y CONTROL ---
Rango [10,60]: OK
Rango [20,40]: FUERA

--- A.8 AMBITOS ANIDADOS ---
Nivel 1: 100 Nivel 2: 200
Nivel 1: 100 Nivel 2: 200 Nivel 3: 300
Nivel 4 (i= 0 ): 400
Nivel 4 (i= 1 ): 401

--- A.9 FUNCIONES EMBEBIDAS ---
Texto: Golampi 2026
Longitud: 12
Subcadena: Golampi
Hora: 2026-04-07 03:17:49
Tipo: string

--- A.10 CASOS LIMITE ---
Array unitario: 42
String vacio longitud: 0
Rango int32: -2147483648 a 2147483647
Comparacion: OK

=== FIN DE CALIFICACION: INTEGRACION ===
```

10. Plan de Sprints — 9 al 29 de Abril 2026

El proyecto se divide en 5 sprints de desarrollo, con 20 días disponibles (9 de abril al 29 de abril). La estrategia es reutilizar la gramática y entornos del Proyecto 1 y construir encima la capa de generación ARM64.

#	Sprint	Fechas	Meta Principal
1	Setup + Reutilización de P1	9-11 Abr (3 días)	ANTLR4 corriendo, semántica base funcionando
2	CodeGen ARM64 - Básico	12-16 Abr (5 días)	Variables, aritmética y <code>fmt.Println</code> generan ARM64 válido
3	Control de Flujo + Arreglos ARM64	17-21 Abr (5 días)	<code>if/else</code> , <code>for</code> , <code>switch</code> , <code>break/continue</code> en ARM64
4	Funciones + Punteros + Builtins	22-26 Abr (5 días)	Llamadas a funciones con convenciones AArch64
5	GUI + Reportes + Integración Final	27-29 Abr (3 días)	App completa lista para calificación

Sprint 1 — Setup + Reutilización de Proyecto 1

9 de abril — 11 de abril (3 días)

Objetivo: El pipeline ANTLR4 -> PHP está funcionando. La gramática del P1 está reutilizada. El análisis semántico base (tabla de símbolos, ámbitos) está operativo. La infraestructura ARM64 tiene su esqueleto.

1	Clonar/copiar el repositorio del Proyecto 1 como base del Proyecto 2
2	Revisar la gramática <code>Golampi.g4</code> del P1 y verificar si necesita ajustes para el P2
3	Regenerar el código PHP con ANTLR4 si la gramática cambia
4	Crear el directorio <code>backend/arm64/</code> con los archivos: <code>ArmEmitter.php</code> , <code>LabelManager.php</code> , <code>RegisterAllocator.php</code> , <code>Assembler.php</code> (esqueletos vacíos)
5	Adaptar <code>backend/interpreter/Visitor.php</code> para agregar campo 'offset' en símbolos (necesario para stack frame ARM64)
6	Adaptar <code>backend/models/Symbol.php</code> : agregar campo offset
7	Crear <code>backend/interpreter/CodeGen.php</code> esqueleto: clase con método <code>generateProgram()</code> que retorna string ARM64
8	Adaptar <code>backend/api/execute.php</code> : incluir <code>arm64_code</code> en el JSON de respuesta
9	Crear directorio <code>output/</code> con <code>.gitkeep</code>
10	Verificar que <code>Assembler.php</code> pueda invocar <code>aarch64-linux-gnu-as</code> y <code>qemu-aarch64</code> en el servidor
11	Prueba de humo: programa vacío genera <code>.section</code> <code>.text</code> sin errores

Sprint 2 — Generación ARM64 Básica: Variables, Aritmética, `Println`

12 de abril — 16 de abril (5 días)

Objetivo: El compilador puede generar código ARM64 válido para: declaración de variables, operaciones aritméticas/relacionales/lógicas, y `fmt.Println` con literales y variables. El archivo `.s` se puede ensamblar y ejecutar con QEMU.

1	Implementar <code>ArmEmitter.php</code> : métodos <code>emit()</code> , <code>emitLabel()</code> , <code>emitSection()</code> , <code>emitGlobal()</code>
2	Implementar <code>LabelManager.php</code> : generador de etiquetas únicas <code>L_0</code> , <code>L_1</code> , etc.
3	Implementar la sección <code>.data</code> para strings literales (<code>fmt.Println</code>)
4	Implementar el prologo/epilogo del <code>_start</code> (main) en <code>CodeGen</code>
5	Implementar generación de código para declaración de variables <code>int32</code> en el stack
6	Implementar asignación de variables (str en el offset correcto del sp)
7	Implementar operaciones aritméticas: <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> (<code>add</code> , <code>sub</code> , <code>mul</code> , <code>sdiv</code>)
8	Implementar operaciones relacionales: <code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> (<code>cmp</code> + flags)
9	Implementar operaciones lógicas: <code>&&</code> , <code> </code> , <code>!</code> con cortocircuito (<code>b.cond</code>)
10	Implementar <code>fmt.Println</code> básico: escribir strings y enteros con <code>syscall write</code> (<code>x8=64</code> , <code>svc #0</code>)
11	Implementar <code>syscall exit</code> al final del programa (<code>x8=93</code> , <code>x0=0</code> , <code>svc #0</code>)
12	Probar: <code>archivo1_basico.go</code> debe compilar y producir la salida correcta via QEMU
13	Implementar operadores de asignación compuesta: <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code>
14	Implementar soporte para tipo <code>bool</code> (0/1) y <code>string</code> en generación de código

Sprint 3 — Control de Flujo + Arreglos en ARM64

17 de abril — 21 de abril (5 días)

Objetivo: El compilador genera ARM64 correcto para: `if/else/else-if`, `switch/case/default`, los tres tipos de `for`, `break`, `continue`, `return`. También soporta arreglos 1D y multidimensionales.

1	Implementar generación de código para <code>if/else</code> : <code>cmp</code> + <code>b.cond</code> + etiquetas <code>L_if_X</code> / <code>L_else_X</code> / <code>L_endif_X</code>
2	Implementar generación de código para <code>if/else-if/else</code> : cadena de etiquetas
3	Implementar generación de código para <code>switch/case</code> : tabla de comparaciones con <code>cmp</code> + <code>b.eq</code>
4	Implementar <code>case</code> con múltiples valores (<code>case 1, 2, 3:</code>)
5	Implementar <code>for</code> clásico (inicialización; condición; post): con etiquetas <code>L_for_X</code> / <code>L_for_end_X</code>
6	Implementar <code>for</code> condicional (tipo <code>while</code>): solo condición
7	Implementar <code>for</code> infinito: <code>b L_for_X</code> sin condición
8	Implementar <code>break</code> : <code>b L_for_end_X</code> (saltar al fin del bucle actual)
9	Implementar <code>continue</code> : <code>b L_for_post_X</code> (saltar al post-increment del bucle actual)
10	Implementar <code>return</code> sin valor: <code>ret</code> / <code>b L_func_end_X</code>
11	Implementar arreglos 1D: reserva de espacio contiguo en el stack, acceso por offset
12	Implementar inicialización de arreglos con literales
13	Implementar acceso y asignación a arreglos: <code>a[i] = v</code> y <code>v = a[i]</code> (<code>ldr/str</code> con desplazamiento variable)
14	Implementar arreglos multidimensionales: cálculo de <code>offset = i*cols + j</code>
15	Probar: <code>archivo2_intermedio.go</code> y <code>archivo4_arreglos1d.go</code> con QEMU

Sprint 4 — Funciones, Punteros y Funciones Embebidas en ARM64

22 de abril — 26 de abril (5 días)

Objetivo: El compilador genera ARM64 completo para funciones con parametros, retorno, multiples retornos, paso por referencia con punteros, hoisting, recursion y todas las funciones embebidas.

1	Implementar prologo de funcion: stp x29, x30, [sp, #-N]!, mov x29, sp, reserva de variables locales
2	Implementar epilogo de funcion: ldp x29, x30, [sp], #N, ret
3	Implementar paso de parametros: primeros 8 en x0-x7, resto en stack
4	Implementar llamada a funcion: bl func_name, manejo de valor de retorno en x0
5	Implementar hoisting: primera pasada registra todas las firmas antes de compilar main
6	Implementar funciones con retorno simple: guardar resultado en x0 antes del epilogo
7	Implementar multiples retornos: x0 y x1 para dos valores (o stack para mas)
8	Implementar punteros: operador & -> cargar direccion de variable (add x0, x29, #offset); operador * -> ldr desde direccion
9	Implementar paso por referencia: parametro puntero, acceso via ldr/str a traves del registro
10	Implementar recursion: las convenciones de llamada ya lo soportan si el prologo/epilogo es correcto
11	Implementar fmt.Println completo: soporte para multiples argumentos variados (int32, string, bool, float32)
12	Implementar len(): para string (contar bytes) y array (tamaño fijo conocido en compile-time)
13	Implementar now(): syscall o call a libc strptime / usar VDSO del kernel
14	Implementar substr(): calcular puntero + offset en memoria del string
15	Implementar typeof(): retornar string literal segun el tipo en compile-time
16	Implementar soporte para float32: registros s0-s31 de punto flotante
17	Probar: archivo3_funciones.go, archivo5_arreglos_ndim.go y archivo6_avanzado.go

Sprint 5 — GUI, Reportes, Documentacion e Integracion Final

27 de abril — 29 de abril (3 días)

Objetivo: La aplicacion es completamente funcional. La GUI muestra el ARM64 generado, los reportes se pueden descargar, y la documentacion esta completa. Lista para calificacion el 29 de abril.

1	Adaptar frontend/index.html: consola ahora muestra 'Consola -Codigo ARM64 Generado' en lugar de output
2	Adaptar frontend/js/api.js: mostrar arm64_code en la consola de la GUI
3	Agregar boton 'Descargar ARM64' en el panel de reportes
4	Implementar frontend/js/reports.js: descarga del .s como archivo
5	Revisar y pulir los botones: Compilar envia codigo y muestra ARM64 o errores
6	Implementar SymbolTableReport.php: generar HTML con tabla de simbolos incluyendo campo offset
7	Implementar ErrorReport.php: generar HTML con tabla de errores
8	Ejecutar todos los archivos de calificacion y verificar outputs con QEMU

9	Corregir bugs encontrados en pruebas finales
10	Redactar README.md con instrucciones de instalacion y uso
11	Redactar gramatica.md con la gramatica formal de Golampi
12	Redactar diagrama-clases.md con el diagrama del compilador
13	Redactar manual-usuario.md con capturas de pantalla
14	Agregar al auxiliar rubenralda como colaborador en el repositorio GitHub
15	Hacer push final y verificar que el repositorio este organizado

11. Checklist de Calificacion

11.1 Funcionalidades Esenciales

#	Requisito	Sprint
1	Gramatica ANTLR4 (.g4) generando lexer y parser PHP	Sprint 1
2	Analisis semantico: tabla de simbolos, ambitos, tipos	Sprint 1
3	Operaciones aritmeticas: +, -, *, /, %	Sprint 2
4	Operaciones relacionales y logicas con cortocircuito	Sprint 2
5	Declaracion de variables: var, :=, const, nil	Sprint 2
6	Tipos: int32, float32, bool, rune, string	Sprint 2
7	fmt.Println generando syscall write ARM64	Sprint 2
8	Sentencia if / else if / else en ARM64	Sprint 3
9	Sentencia switch / case / default en ARM64	Sprint 3
10	Bucle for: clasico, condicional, infinito en ARM64	Sprint 3
11	Sentencias break, continue, return en ARM64	Sprint 3
12	Arreglos 1D y multidimensionales en ARM64	Sprint 3
13	Funciones con parametros y retorno en ARM64	Sprint 4
14	Multiples valores de retorno en ARM64	Sprint 4
15	Paso por referencia con punteros (& y *) en ARM64	Sprint 4
16	Hoisting de funciones	Sprint 4
17	Builtins: fmt.Println, len, now, substr, typeof en ARM64	Sprint 4
18	Codigo ARM64 generado (.s) descargable desde la GUI	Sprint 5
19	GUI con editor de codigo y consola que muestra ARM64	Sprint 5
20	Reporte de errores (lexico, sintactico, semantico)	Sprint 5
21	Reporte tabla de simbolos con offset de stack	Sprint 5
22	Repositorio GitHub con auxiliar rubenralda	Sprint 5
23	El codigo ARM64 se ejecuta correctamente via QEMU	Todos

11.2 Convenciones AArch64 Obligatorias

- x0-x7 para primeros 8 parametros
- x0 para valor de retorno
- x29 (fp) como frame pointer
- x30 (lr) como link register
- sp manejado explicitamente con prologo y epilogo
- Codigo ARM64 comentado en secciones criticas

- Archivo de salida con extension .s

11.3 Entregables Finales

Entregable	Descripcion
Prototipo funcional	Version ejecutable del compilador que genera ARM64 valido
Gramatica ANTLR4 (.g4)	Archivo de gramatica formal del lenguaje Golampi
Codigo PHP (Visitor + CodeGen)	Implementacion del compilador con generacion ARM64
Interfaz grafica (GUI)	Editor web con consola ARM64 y panel de reportes
Documentacion tecnica	Markdown con gramatica, diagrama de clases, flujo
Manual de usuario	Guia de instalacion, uso e interpretacion de reportes
Repositorio GitHub	Codigo fuente organizado; rubenralda como colaborador